

EGCO 425 – Chapter 10

Clustering Methods

K-Means

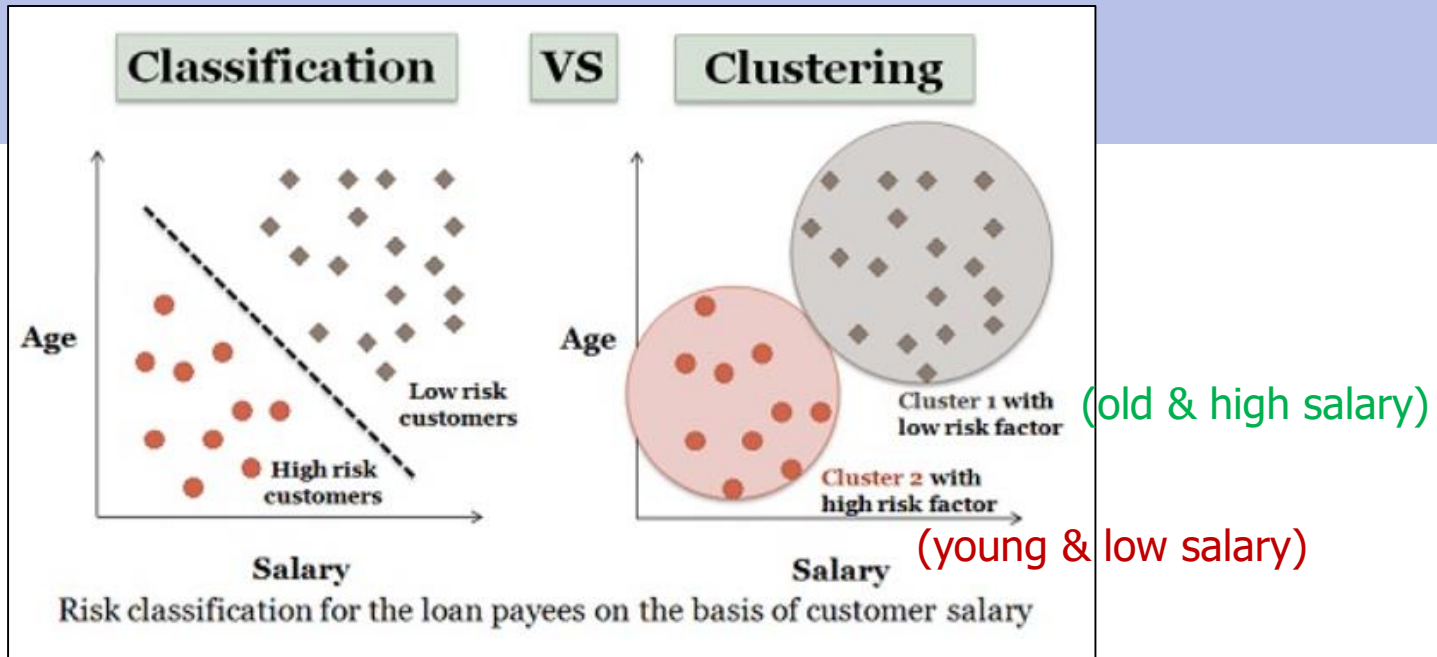
Expectation-Maximization (EM)

References

- [1] Tan et al., Introduction to Data Mining (Chapters 2, 8)
- [2] Bramer, Principles of Data Mining (Chapter 14)
- [3] Kotu & Deshpande, Data Science: Concepts and Practice (Chapter 7)
- [4] RapidMiner Doc, <https://docs.rapidminer.com/latest/studio/operators/>

Clustering

- ⊕ Find meaningful groups (i.e. clusters) in data
 - ▶ It is not to predict class variable, but to simply capture the possible natural groupings
- ⊕ Difference between classification and clustering
 - ▶ **Classification is supervised learning** because target class is specified. We use attributes as independent variables to predict class
 - ▶ **Clustering is unsupervised learning** because there is no target class. All attributes are treated as independent variables
- ⊕ In RapidMiner, clustering task is called segmentation



Classification	Clustering
Independent variables = age, salary	Independent variables = age, salary
Classes = high risk, low risk	Classes = none
Find model (e.g. linear function of age and salary) to predict class	Group similar records
Use the model to predict new cases	Build cluster profiles by observing the attributes of their members

Clustering Purposes

- ⊕ To understand the data
 - ▶ Clustering is not about prediction
 - ▶ But to understand how objects are similar & grouped together, i.e. to make sense of the group as a whole
- ⊕ As preprocessing step before other data analysis
 - ▶ Reducing #attributes → use cluster membership as 1 nominal attribute instead of m original attributes
 - E.g. in Slide 3, we can use risk factor (1 attribute) instead of age and salary (2 attributes) in further data analysis
 - ▶ Reducing #records → use cluster prototypes instead of raw data
 - E.g. 100 clusters are discovered from 1 million records. We can use these clusters as individual records in further data analysis

Chapter Outline

⊕ K-Means and K-Medoids

- ▶ Clustering steps
- ▶ Centroids
- ▶ Sum of squared error (SSE), Davis-Bouldin index (DBI)

⊕ Expectation-maximization (EM)

- ▶ Mixture models & assumption
- ▶ Clustering steps, likelihood, log-likelihood

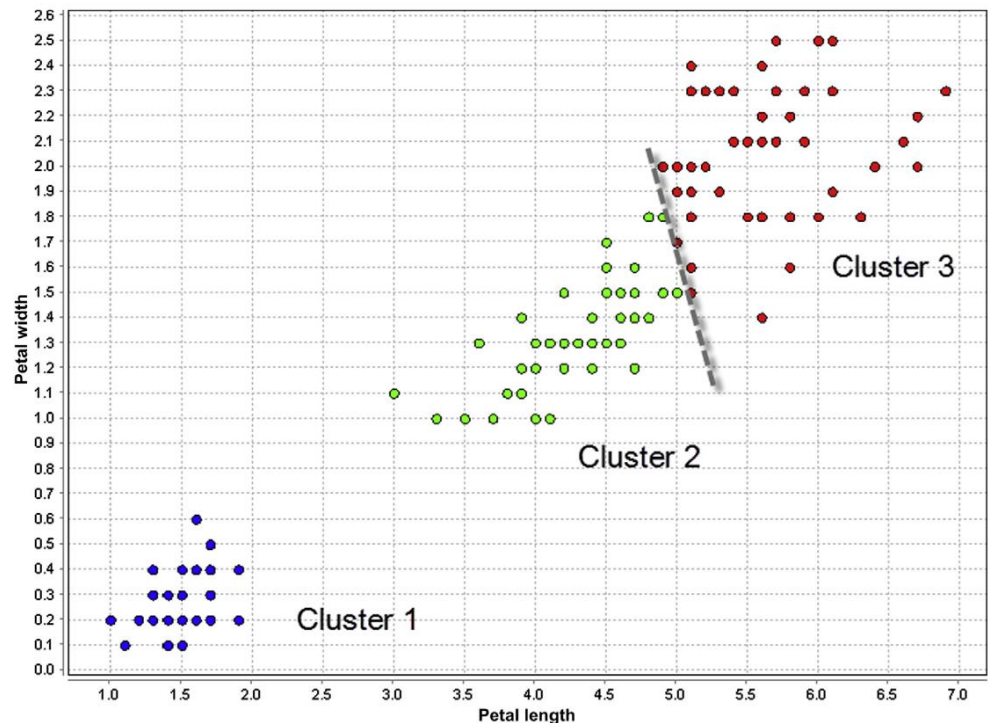
⊕ Visualization → <https://educlust.dbvis.de/>

⊕ Evaluation & interpretation

- ▶ Unsupervised vs. supervised approaches
- ▶ Cluster profiles

Types of Cluster Membership

- ✦ Different clustering methods give different types of clusters, depending on how cluster membership is assigned to each object
- ✦ Exclusive or strict partitioning clusters
 - ▶ Each object belongs exclusively to 1 cluster
 - ▶ The most common type of clusters



⊕ Overlapping clusters

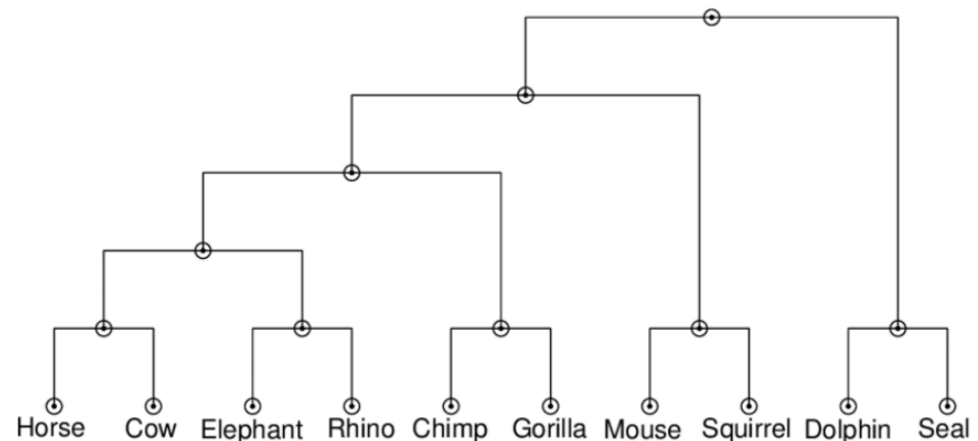
- ▶ An object can simultaneously belong to multiple clusters
- ▶ E.g. a person (part-time lecturer & Ph.D. student) can be in both teacher and student clusters

⊕ Fuzzy or probabilistic clusters

- ▶ Each object belongs to all clusters with different degrees of membership or probabilities from 0 to 1

⊕ Hierarchical clusters

- ▶ A cluster can be nested inside the others
- ▶ So an object can belong to multiple clusters



Clustering Approaches

⊕ Prototype-based clustering

- ▶ Each cluster is represented by a central object, called prototype or centroid (since it is usually the center of cluster)
- ▶ Compare each object with all cluster centroids and assign the object to the closest one → using similarity or distance function
- ▶ Cluster shape is typically globular
- ▶ Clustering methods → K-Means, K-Medoids

⊕ Density-based clustering

- ▶ A cluster is defined as a dense region where objects are concentrated, surrounded by a low-density area where objects are sparse
- ▶ Objects in low-density area are discarded as noises
- ▶ Clustering methods → DBSCAN

⊕ Hierarchical clustering

- ▶ Most similar objects are grouped as innermost clusters. Then similar clusters are merged to form bigger ones
- ▶ Output = dendrogram. Clusters are extracted by cutting dendrogram
- ▶ Clustering methods → agglomerative clustering

⊕ Probability-based clustering

- ▶ A cluster of objects (or samples) is assumed to be drawn from a certain population. So it is represented by distribution model of that population, e.g. normal distribution with μ and σ^2 parameters
- ▶ Each object is assigned to the most likely cluster
- ▶ Clusters can have any shape, any density
- ▶ Clustering methods → EM

K-Means

- ⊕ Simplest and most commonly used clustering algorithm
- ⊕ Find K clusters that are represented by centroids

Select K data points as initial centroids

For each cycle {

For each data point X {

1. Calculate the distance from X to each centroid
2. Assign X to the closest centroid

}

Complete 1 cycle i.e. all points are put in clusters → update centroids

Calculate SSE, DBI

} until stopping criteria e.g. centroids are stable, SSE or DBI is stable,
#cycles completed

Measurements

- ⊕ Popular **distance function = Euclidean** (see Chapter 5)
 - ▶ Data preprocessing such as normalization may be applied to handle types, scales, and weights of attributes
- ⊕ **Centroid = mean of all points in the cluster**
 - ▶ If attributes are $A_1, A_2, \dots, A_m$

$$\text{centroid} = (\text{mean of } A_1, \text{mean of } A_2, \dots, \text{mean of } A_m)$$

- ⊕ **SSE = sum of squared error**

$$SSE = \sum_{\forall \text{ clusters}} \left(\sum_{\forall \text{ points in cluster}} (d \text{ from point to centroid})^2 \right)$$

- ▶ Objective = minimize SSE
- ▶ Average within-cluster distance = $\sqrt{SSE}/K$

Davies-Bouldin Index (DBI)

✦ R_{ij} = within-between distance ratio between clusters i and j

$$R_{ij} = \frac{\text{Avg within cluster distance of } i + \text{Avg within cluster distance of } j}{\text{distance between centroids } i \text{ and } j}$$

- ▶ We want this ratio to be small → objects in the same clusters are close to each other (cohesion) and clusters are well separated (separation)
- ▶ Worst-case scenario for cluster i is to consider the maximum R_{ij}

$$D_i = \max \{ R_{ij} \} ; \text{ considering all other clusters } j$$

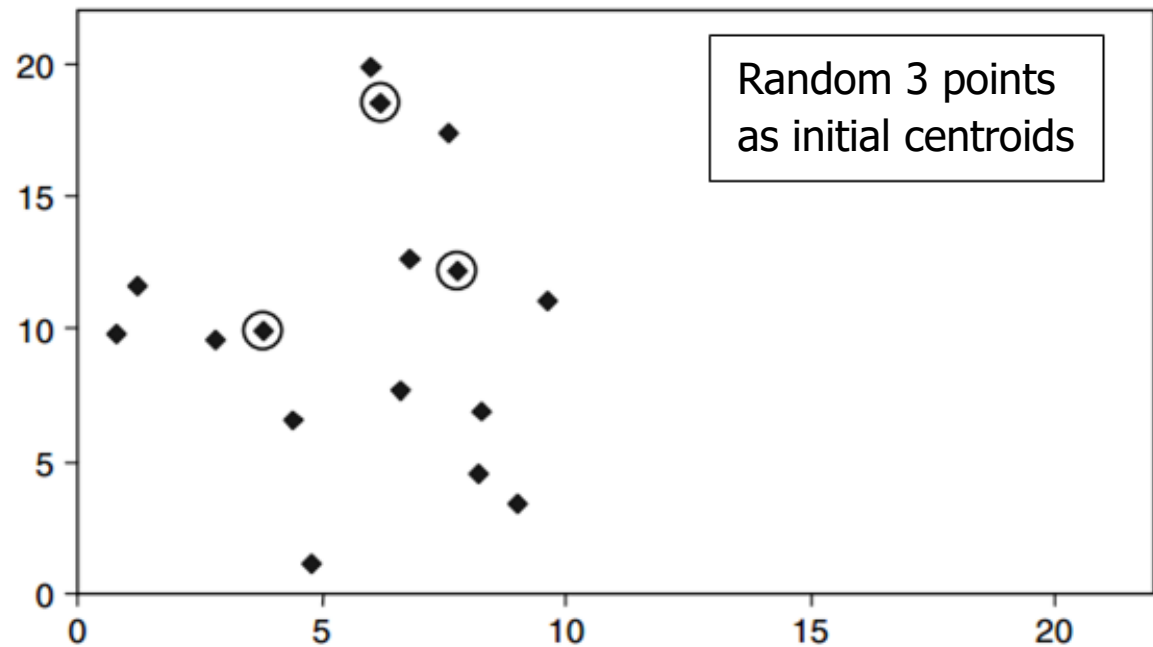
✦ For K clusters

$$DBI = \frac{\text{sum of } D_i \text{ for all clusters}}{K}$$

- ▶ Objective = minimize DBI

x	y
6.8	12.6
0.8	9.8
1.2	11.6
2.8	9.6
3.8	9.9
4.4	6.5
4.8	1.1
6.0	19.9
6.2	18.5
7.6	17.4
7.8	12.2
6.6	7.7
8.2	4.5
8.4	6.9
9.0	3.4
9.6	11.1

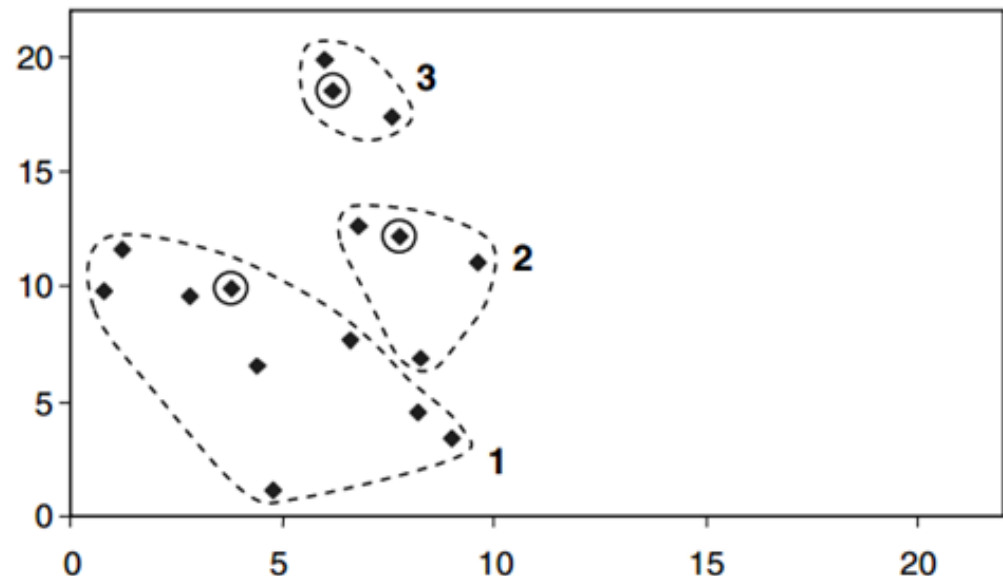
Given 16 records, 2 attributes



Euclidean distance from
each point to centroid

$$\sqrt{(6.8 - 3.8)^2 + (12.6 - 9.9)^2} = 4.0$$

x	y	$d1$	$d2$	$d3$	cluster
6.8	12.6	4.0	1.1	5.9	2 ●
0.8	9.8	3.0	7.4	10.2	1
1.2	11.6	3.1	6.6	8.5	1
2.8	9.6	1.0	5.6	9.5	1
3.8	9.9	0.0	4.6	8.9	1
4.4	6.5	3.5	6.6	12.1	1
4.8	1.1	8.9	11.5	17.5	1
6.0	19.9	10.2	7.9	1.4	3
6.2	18.5	8.9	6.5	0.0	3
7.6	17.4	8.4	5.2	1.8	3
7.8	12.2	4.6	0.0	6.5	2 ●
6.6	7.7	3.6	4.7	10.8	1
8.2	4.5	7.0	7.7	14.1	1
8.4	6.9	5.5	5.3	11.8	2 ●
9.0	3.4	8.3	8.9	15.4	1
9.6	11.1	5.9	2.1	8.1	2 ●



Update centroid of cluster 2 (K-Means)

$$x = (6.8 + 7.8 + 8.4 + 9.6) / 4 = 8.2$$

$$y = (12.6 + 12.2 + 6.9 + 11.1) / 4 = 10.7$$

Centroid is an imaginary point

For K-Medoids

Calculate average distance from each point to
all other points in cluster 2

New centroid = point with min avg distance

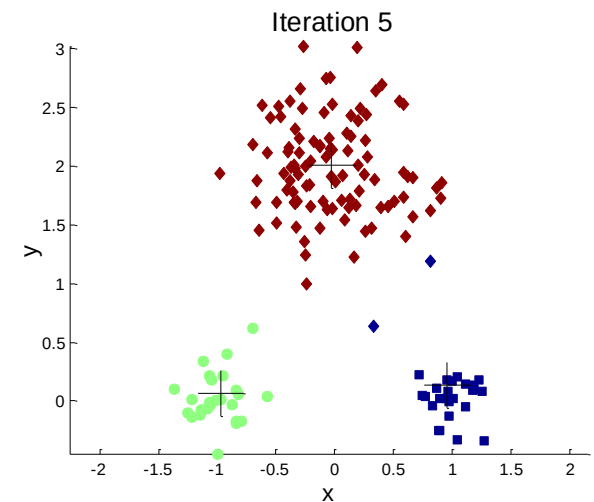
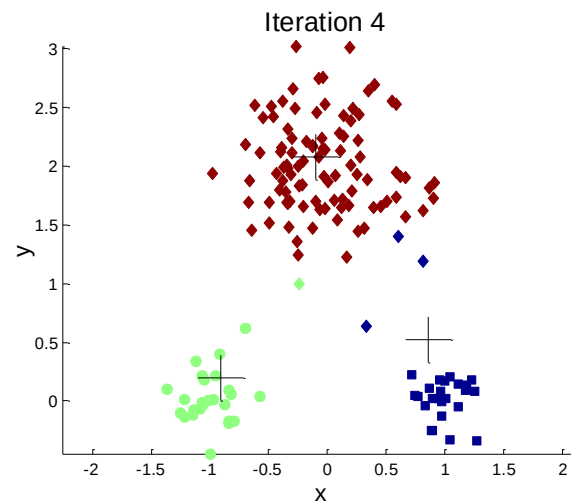
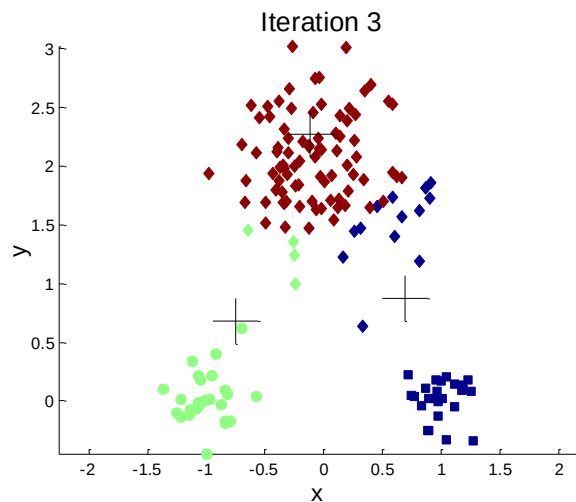
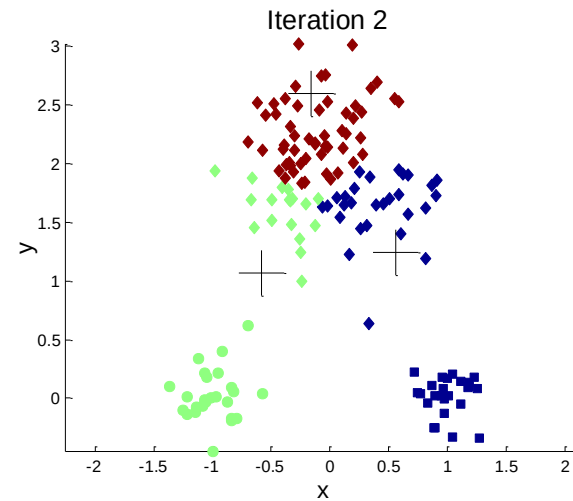
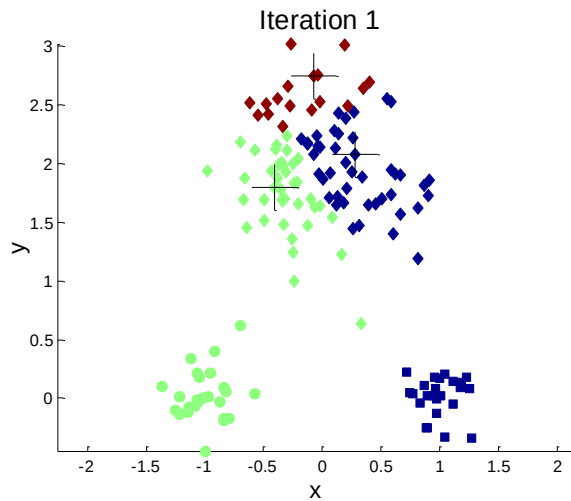
Centroid is the best actual point in cluster

K-Means vs. K-Medoids

Object	X	Y	Distance to other points in cluster			
1	6.8	12.6	d12 = 1.1	d13 = 5.9	d14 = 3.2	mean = 3.4
2	7.8	12.2	d21 = 1.1	d23 = 5.3	d24 = 2.1	mean = 2.8
3	8.4	6.9	d31 = 5.9	d32 = 5.3	d34 = 4.4	mean = 5.2
4	9.6	11.1	d41 = 3.2	d42 = 2.1	d43 = 4.4	mean = 3.2
Centroid in K-Means	8.2	10.7	object 2 = centroid (aka medoid) in K-Medoids			

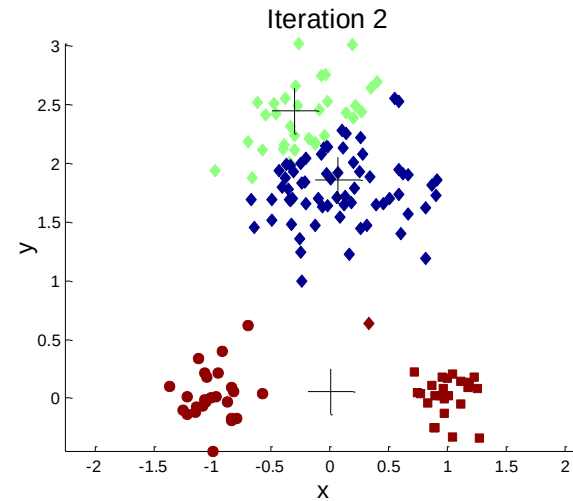
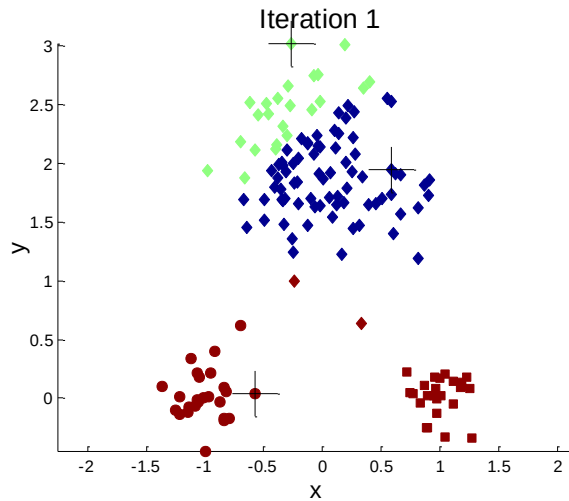
- ⊕ K-Means is sensitive to outliers because outliers pull centroids away from where they should be, leading to distorted results
- ⊕ K-Medoids is more robust to outliers because outliers are far from other points & will never be picked as centroids

Initial Centroids (1)

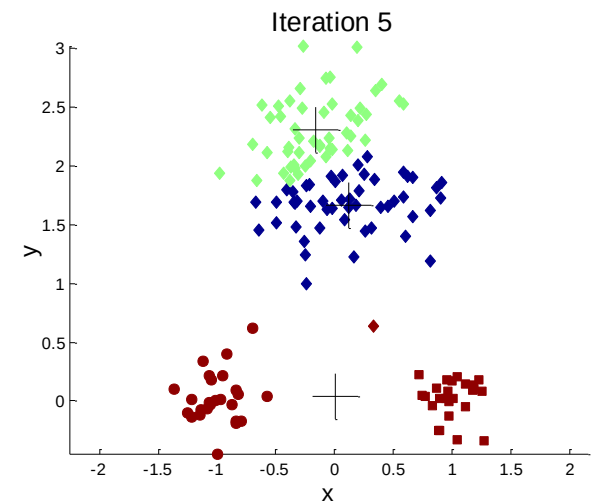
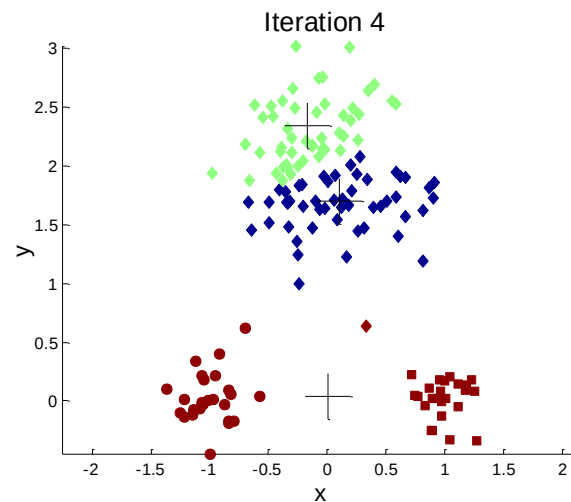
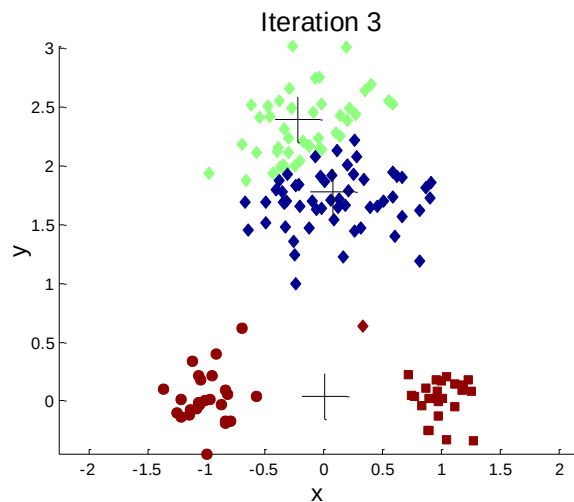


Good clusters

Initial Centroids (2)



Not good clusters



Choosing Initial Centroids

- ⊕ Clustering results are sensitive to initial centroids
 - ▶ Sometimes, initial random centroids adjust themselves in the right way. Sometimes, they do not, especially when K is large

Possible solutions

- ⊕ Try different sets of initial centroids
 - ▶ Then pick the best result, with lowest SSE
 - ▶ But search space is infinite & optimal result is not guaranteed
- ⊕ Farthest points selection
 - ▶ First centroid = a random point or the centroid of the whole data set
 - ▶ Next centroid = point farthest from all previous centroids
 - ▶ Centroids are well separated, but outliers may be picked as centroids

⊕ K-Means++

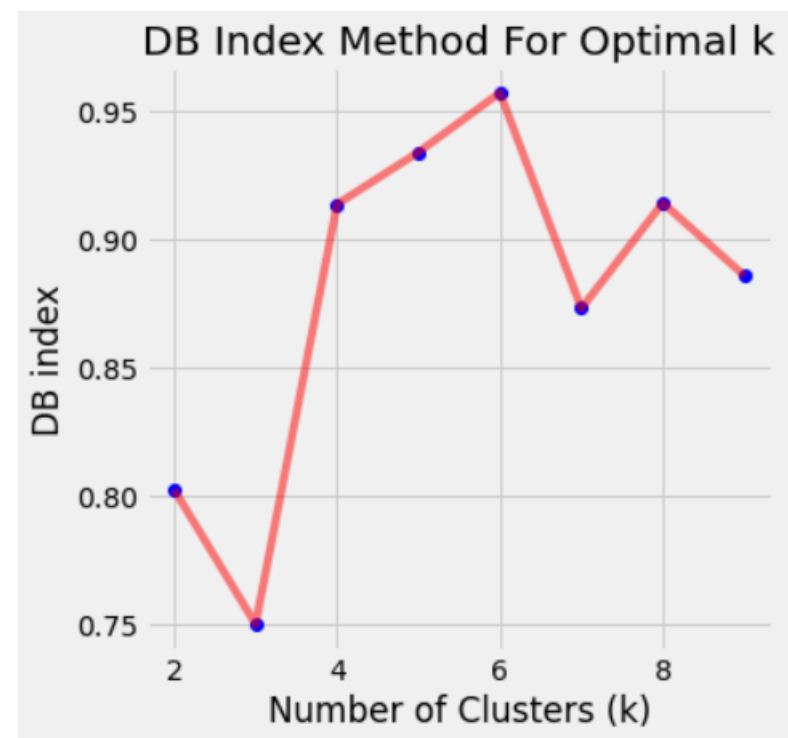
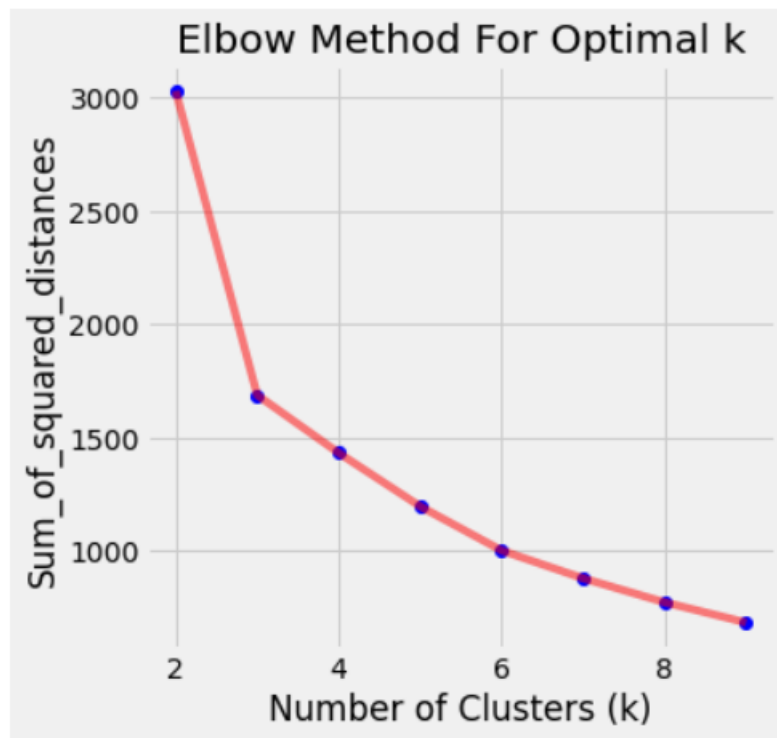
- ▶ Concept is similar to farthest points selection
- ▶ But use probability instead of distance → if a point is very far from all previous centroids, its probability will be very high
- ▶ Next centroid = point with the highest probability

⊕ Post-processing to fixup the results

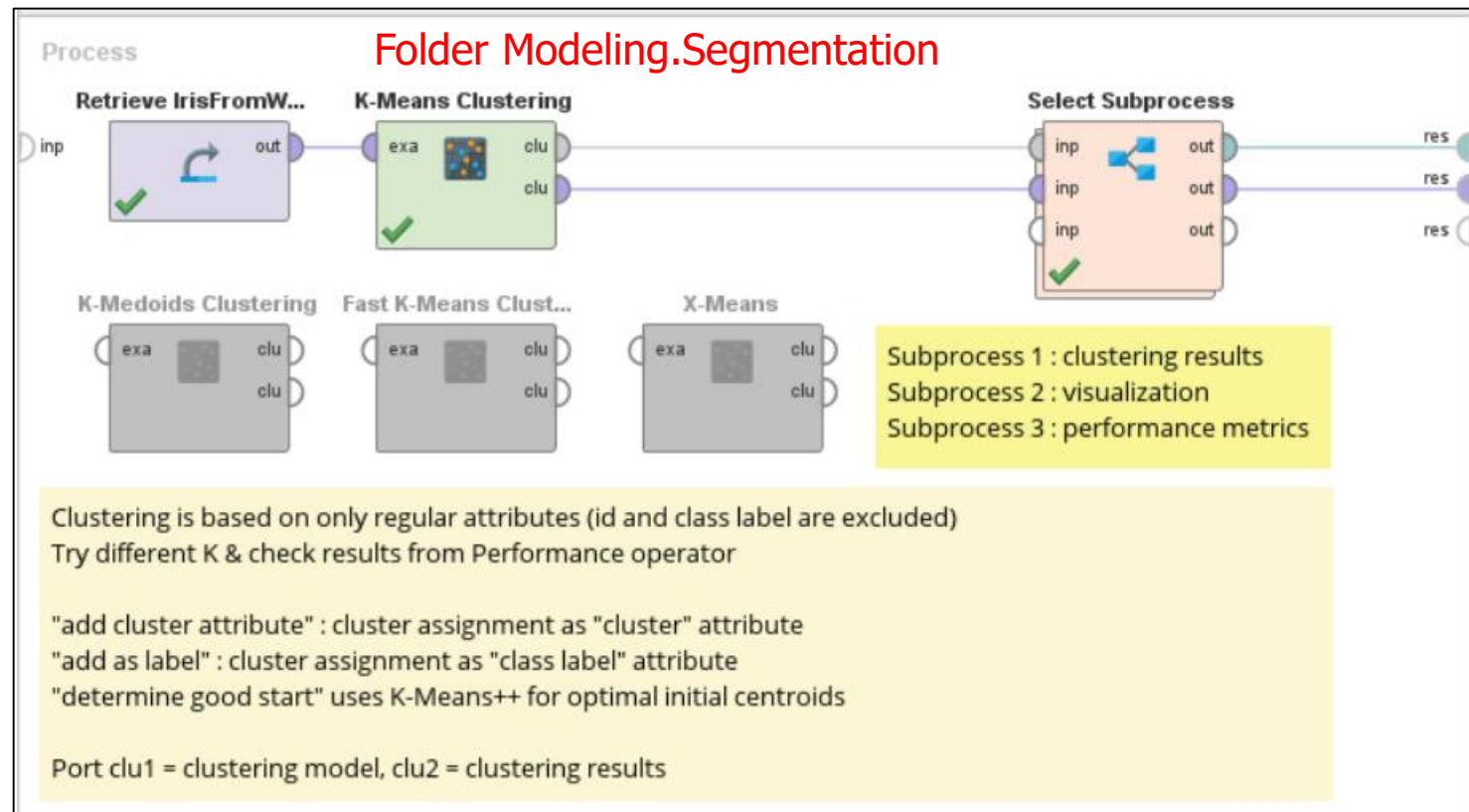
- ▶ Use any method and check results manually
- ▶ Determine whether to split, merge, or remove clusters
 - To get lower SSE
 - For better analysis (some clusters may have very similar profile)
 - Some small clusters may be outliers

Choosing Number of Clusters (K)

- ✦ Try different values of K to get different clustering results
 - ▶ Elbow method plot SSE against K and pick K at the elbow
 - ▶ DBI method plot DBI against K and pick K at minimum DBI

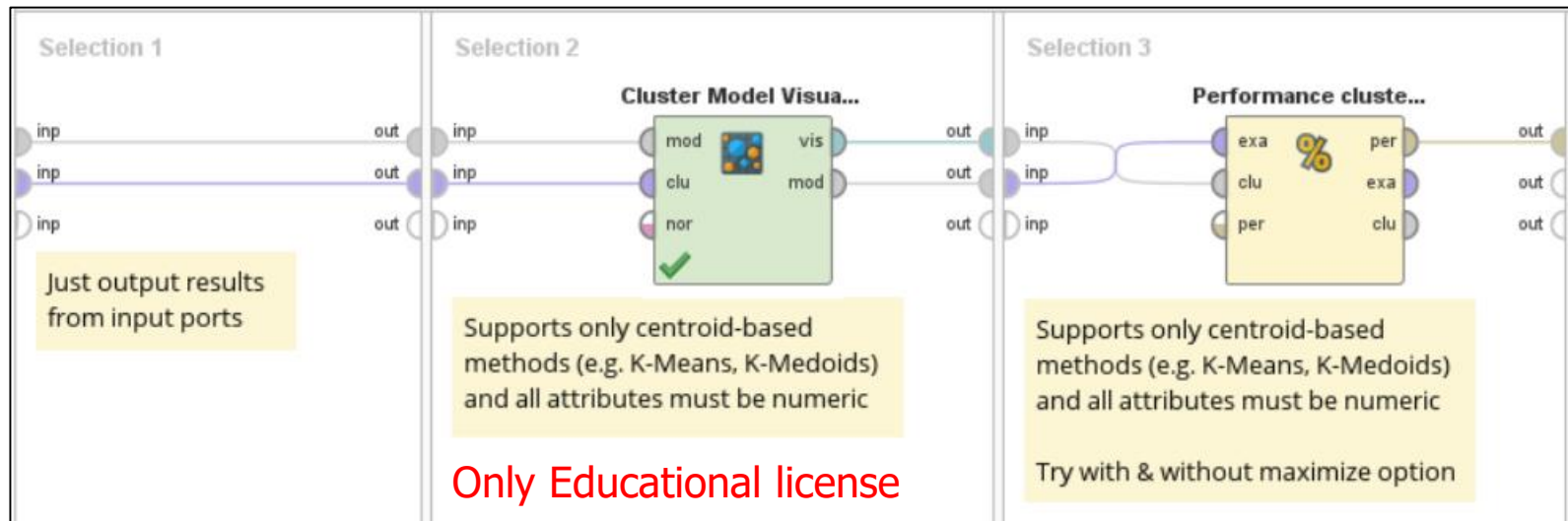


Example (1) : K-Means

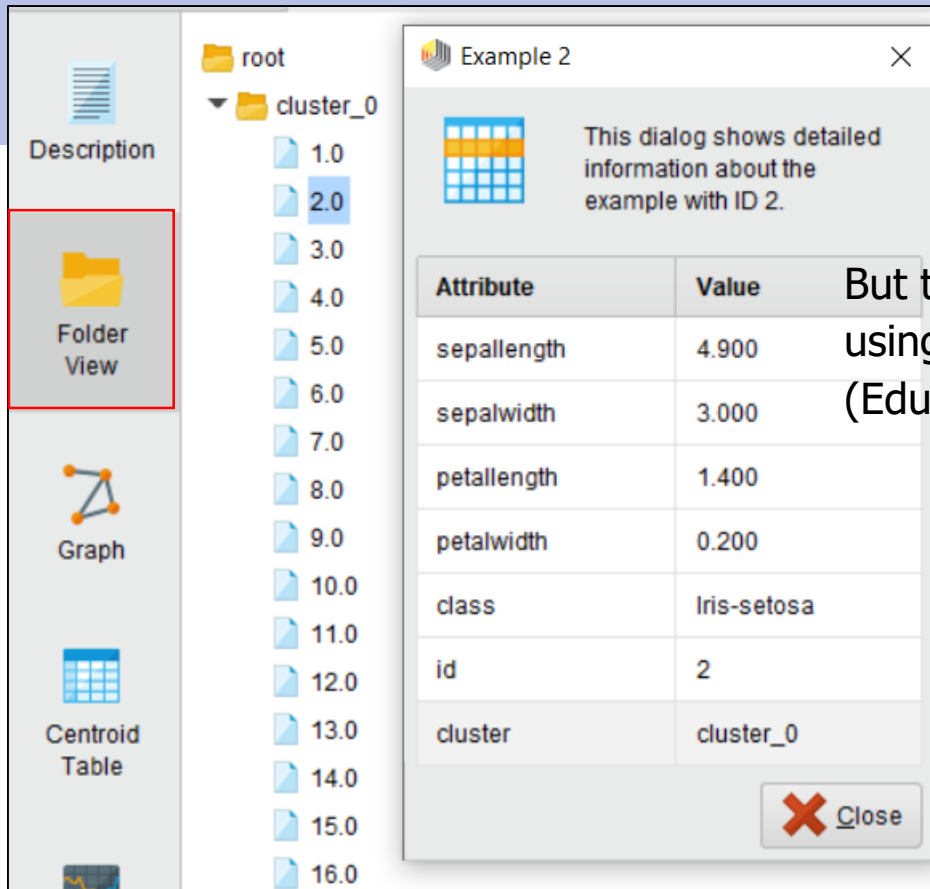


- ⊕ Fast K-Means = another implementation of K-Means
 - ▶ Faster, but require more memory
- ⊕ X-Means = wrapper for K-Means to automatically discover K
 - Run K-Means for the whole data set using $K = \min K$
 - Loop {
 - Split each resulting (i.e. parent) cluster by running local K-Means
 - Between splitting and not splitting → keep option with best BIC
 - } until reaching max K or best BIC
- ⊕ Bayesian information criterion (BIC)
 - ▶ = likelihood (i.e. goodness) of model – model complexity penalty
 - ▶ More centroids or parameters → higher complexity → higher penalty
 - ▶ *Some papers use model complexity as the first term*

Folder Validation.Performance.Segmentation

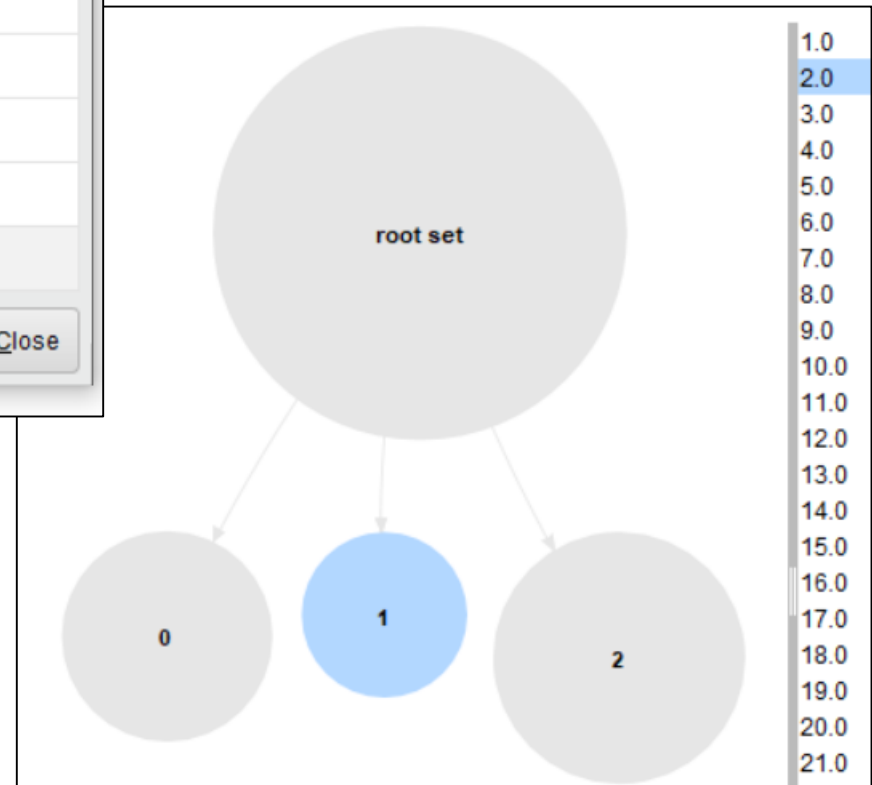


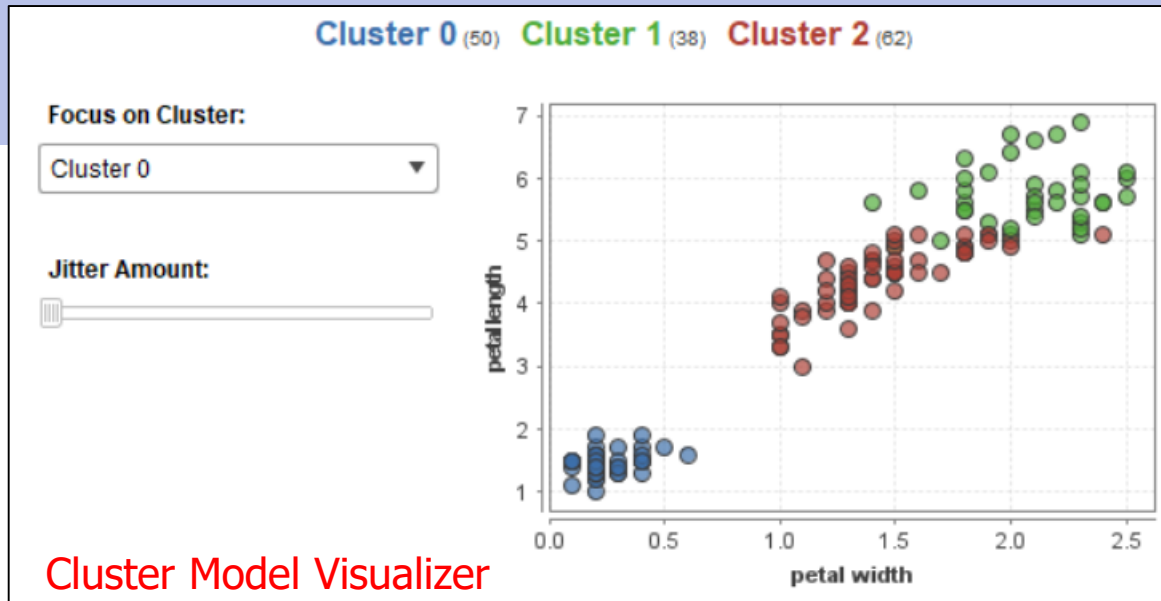
Row No.	id	class	cluster	sepallength	sepalwidth	petallength	petalwidth
1	1	Iris-setosa	cluster_0	5.100	3.500	1.400	0.200
2	2	Iris-setosa	cluster_0	4.900	3	1.400	0.200
3	3	Iris-setosa	cluster_0	4.700	3.200	1.300	0.200
4	4	Iris-setosa	cluster_0	4.600	3.100	1.500	0.200
5	5	Iris-setosa	cluster_0	5	3.600	1.400	0.200



But this dialog is shown only when using Cluster Model Visualizer (Educational license)

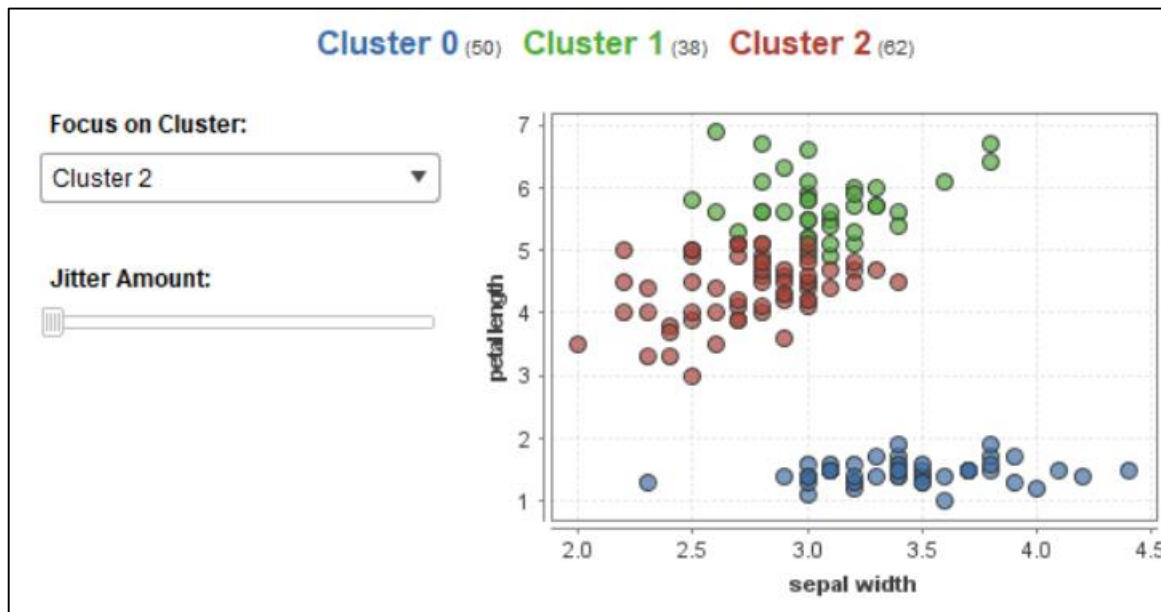
Both Free and Education licenses give the same basic elements for cluster model





Cluster Model Visualizer
(Educational license)

Dominant features of cluster 0 that clearly separate its members from the others = petal width & petal length



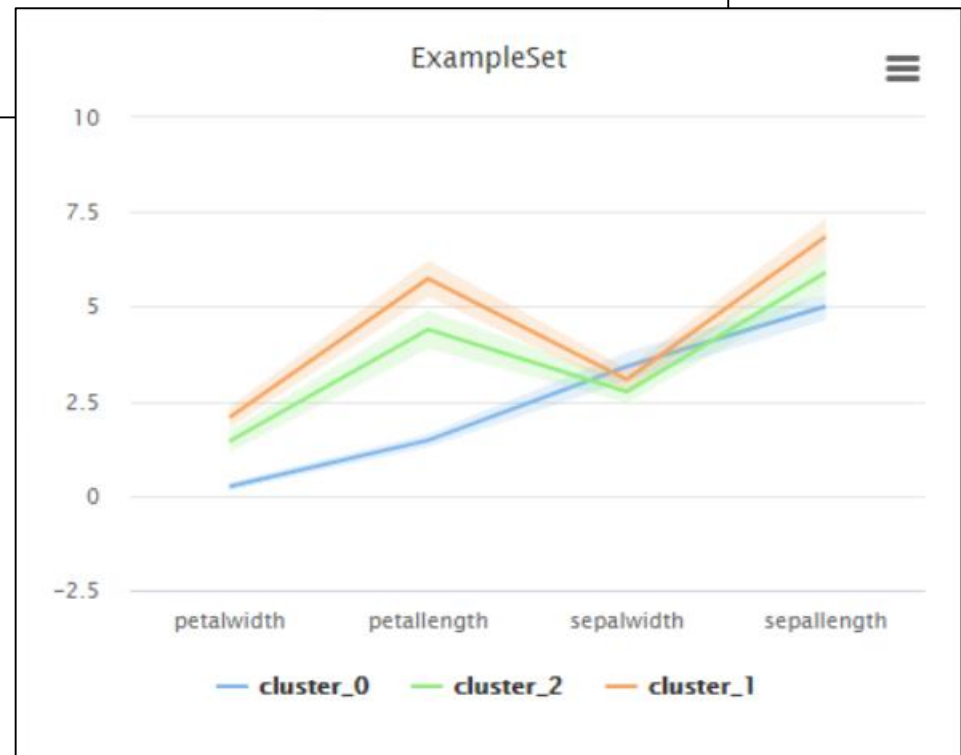
Dominant features of cluster 2. The separation is not as clear as those of cluster 0

For Free license, we can run visualization on clustering result instead



Creating our own visualization

- **Scatter plot** shows how data are clustered
- **Deviation chart** shows overview of cluster profiles (mean & SD of all attributes)



PerformanceVector

```
PerformanceVector:
Avg. within centroid distance: 0.526
Avg. within centroid distance_cluster_0: 0.305
Avg. within centroid distance_cluster_1: 0.628
Avg. within centroid distance_cluster_2: 0.642
Davies Bouldin: 0.662
```

With maximize option (in Performance operator). We want all values to be as low as possible e.g. DBI 0.2 is better than 0.6

PerformanceVector

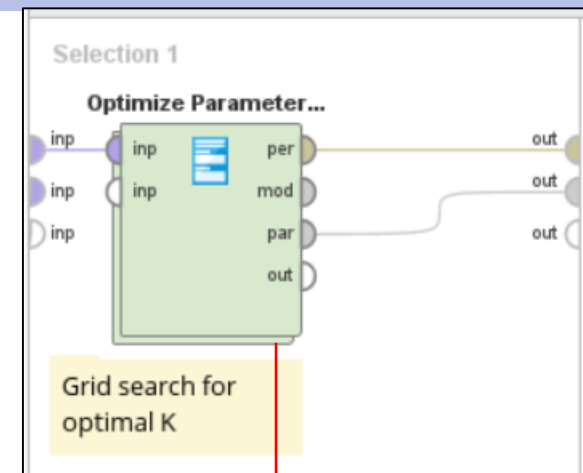
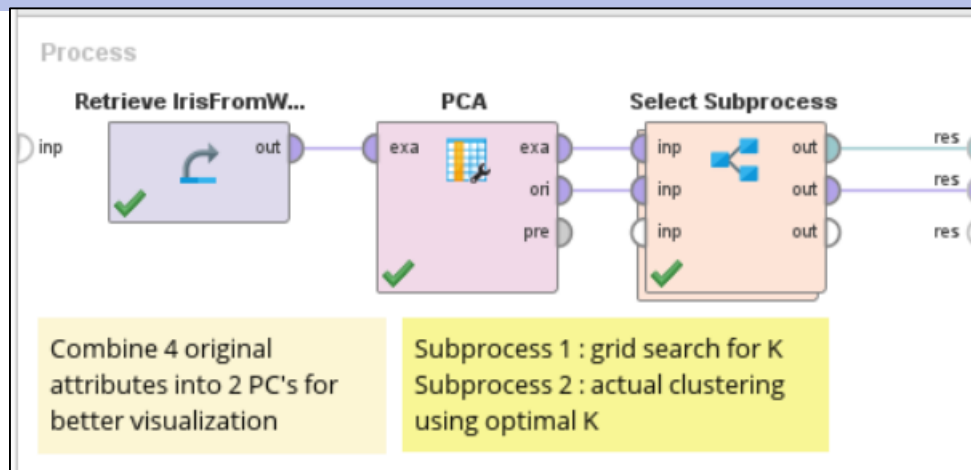
```
PerformanceVector:
Avg. within centroid distance: -0.526
Avg. within centroid distance_cluster_0: -0.305
Avg. within centroid distance_cluster_1: -0.628
Avg. within centroid distance_cluster_2: -0.642
Davies Bouldin: -0.662
```

Without maximize option → multiply -1 to all values to make the comparison easier e.g. DBI -0.2 is higher than -0.6 so it is better

Trying different K

K	2	3	4	5	6	7	8	9	10
(-1) DBI	-0.405	-0.662	-0.781	-0.816	-0.903	-0.995	-0.969	-0.999	-0.975
Avg. distance	1.016	0.526	0.382	0.310	0.261	0.230	0.199	0.189	0.172

Example (2) : K-Means with PCA



ParameterSet

Parameter set:

Performance:

PerformanceVector [

-----Avg. within centroid distance: -0.914

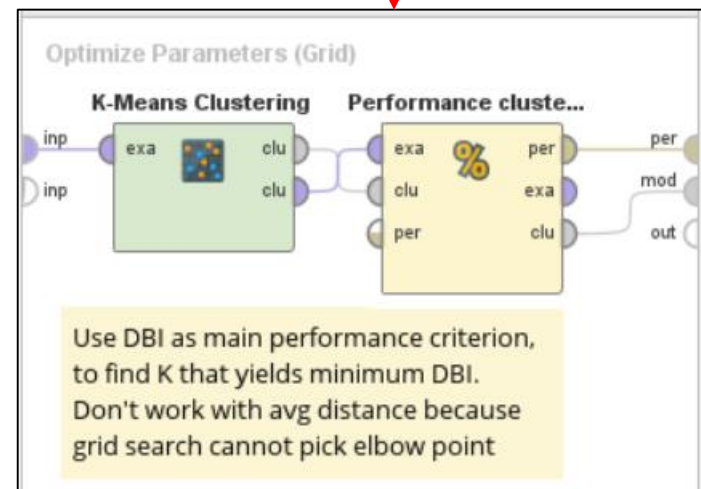
-----Avg. within centroid distance_cluster_0: -0.498

-----Avg. within centroid distance_cluster_1: -1.142

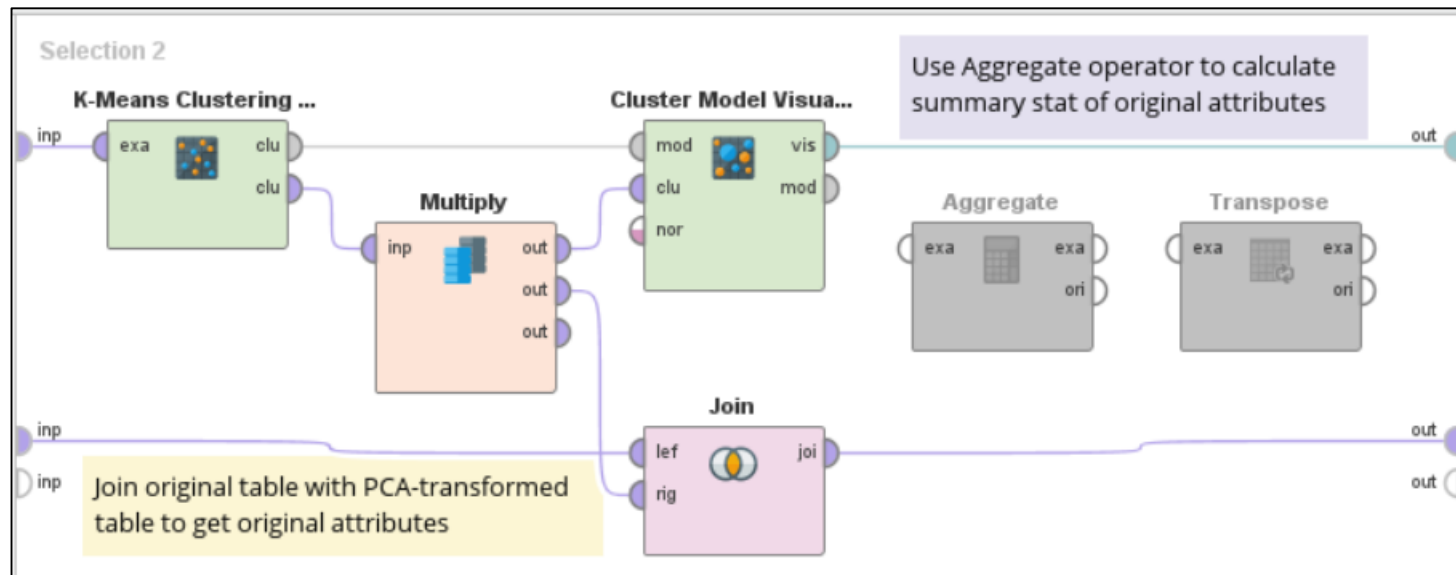
*****Davies Bouldin: -0.371

]

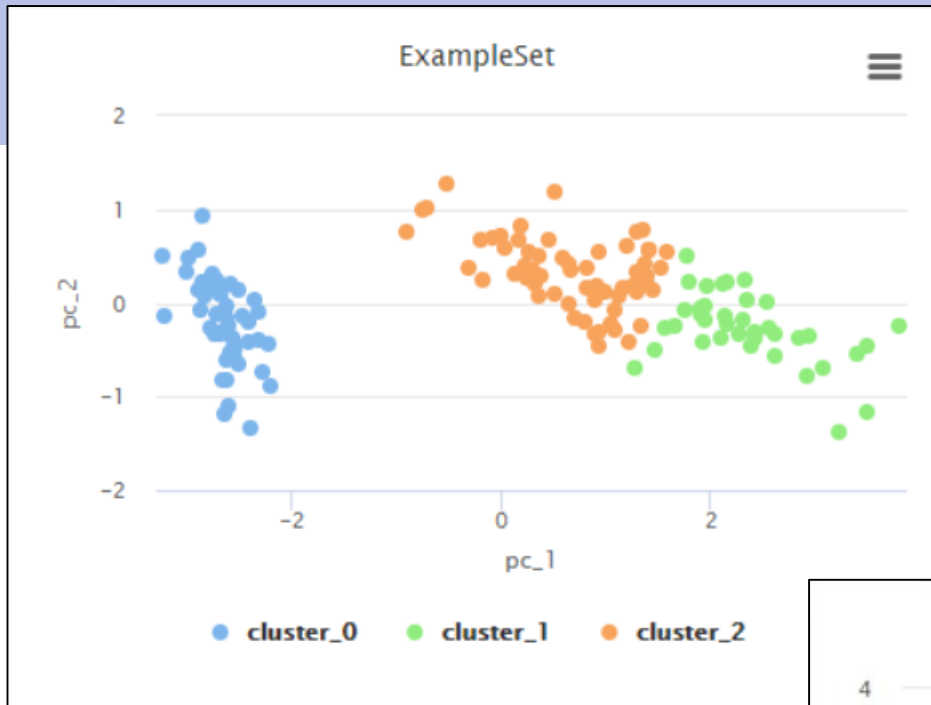
K-Means Clustering.k = 2



Folders Blending.Table.Grouping and Blending.Table.Rotation

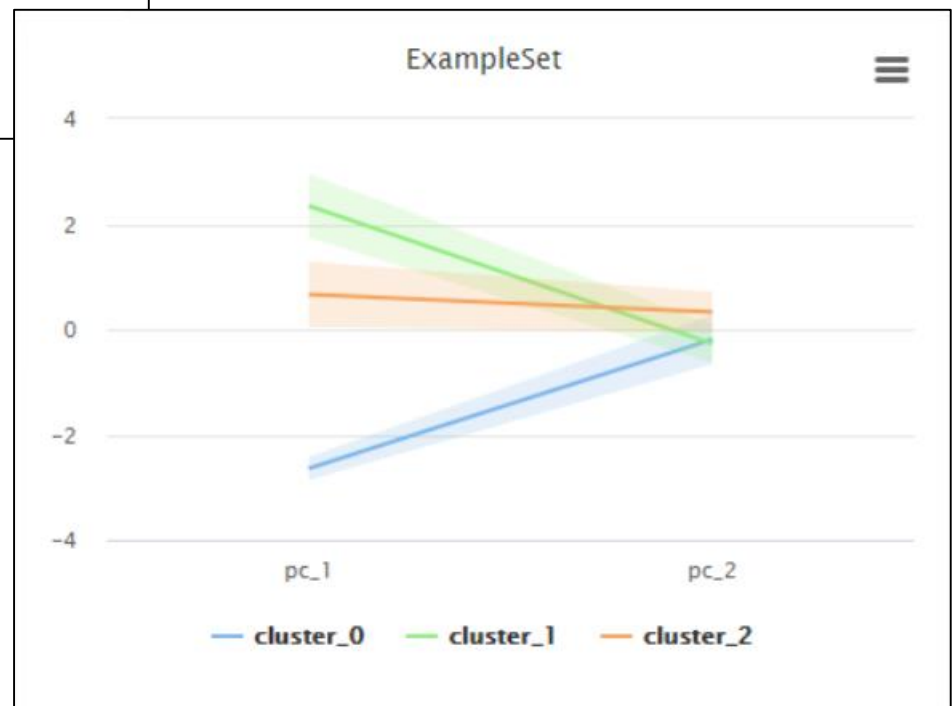


Optimal K from grid search = 2. But we use K = 3 in order to compare results with those from previous example



PCA allows better visualization (to see how data points are clustered)

But cluster profiles based on PC's are difficult to understand → check original attributes

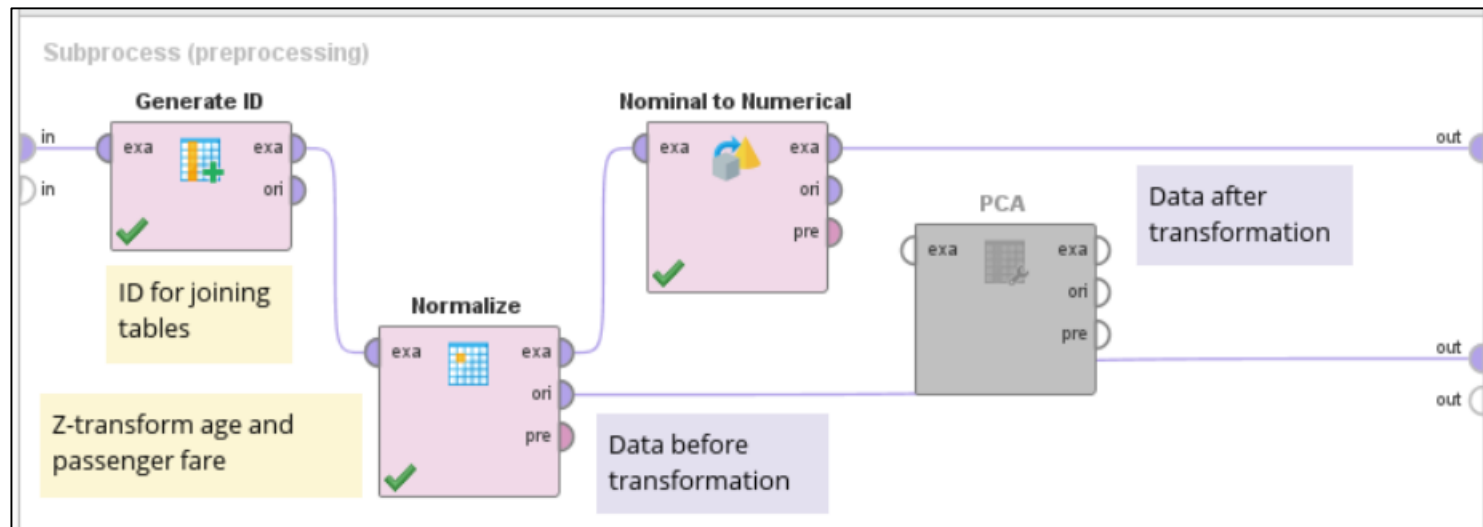
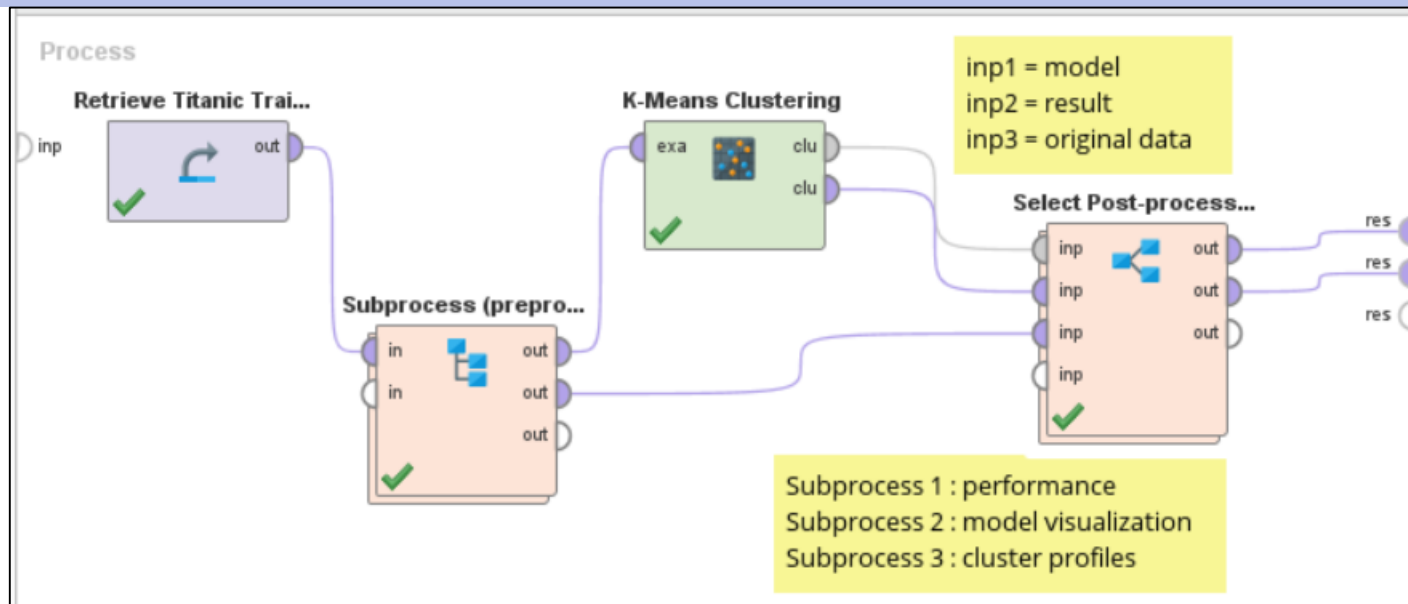


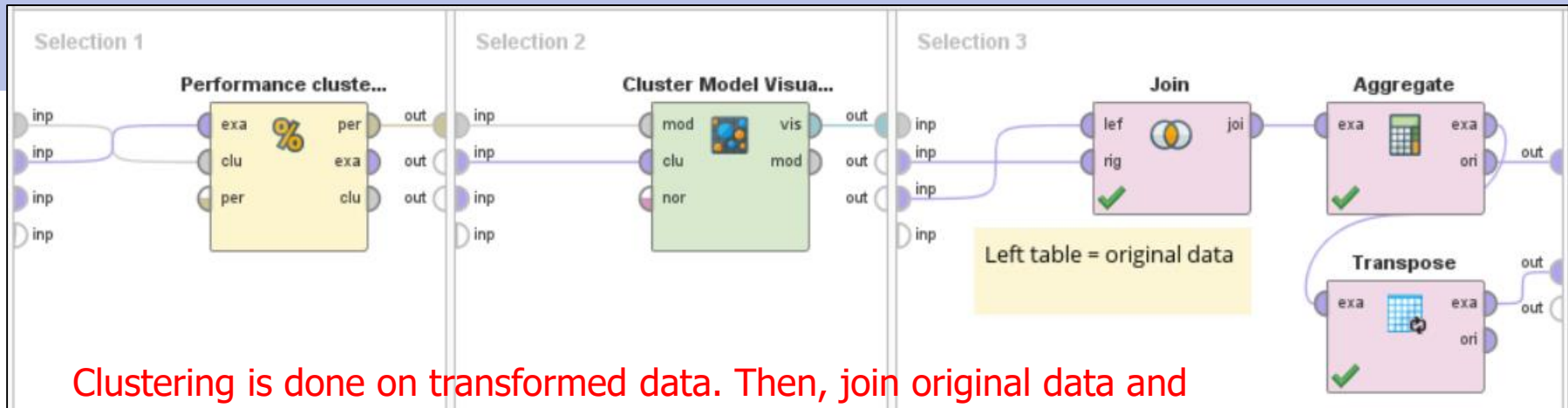
Row No.	id	class	cluster	sepalength	sepalwidth	petallength	petalwidth	pc_1	pc_2
1	1	Iris-setosa	cluster_0	5.100	3.500	1.400	0.200	-2.684	-0.327
2	2	Iris-setosa	cluster_0	4.900	3	1.400	0.200	-2.715	0.170
3	3	Iris-setosa	cluster_0	4.700	3.200	1.300	0.200	-2.890	0.137
4	4	Iris-setosa	cluster_0	4.600	3.100	1.500	0.200	-2.746	0.311
5	5	Iris-setosa	cluster_0	5	3.600	1.400	0.200	-2.729	-0.334

Row No.	id	att_1	att_2	att_3
1	average(petallength)	1.464	5.715384615...	4.388524590...
2	average(petalwidth)	0.243999999...	2.053846153...	1.434426229...
3	average(sepalength)	5.005999999...	6.853846153...	5.883606557...
4	average(sepalwidth)	3.418000000...	3.076923076...	2.740983606...
5	count(cluster)	50.0	39.0	61.0
6	cluster	cluster_0	cluster_1	cluster_2

Describe the profile of each cluster by using summary statistics on original attributes

Example (3) : K-Means with nominal attributes





Clustering is done on transformed data. Then, join original data and clustering results. So we can build cluster profiles based on original data

Attribute (transformed)	cluster_0	cluster_1	cluster_2
Passenger Class = First	0.139	0.980	0.053
Passenger Class = Second	0.229	0.020	0.271
Passenger Class = Third	0.632	0	0.677
Sex = Female	0.260	0.557	0.556
Sex = Male	0.740	0.443	0.444
Age	-0.002	0.942	-1.044
Passenger Fare	-0.365	1.596	-0.048
No of Siblings or Spouses on Board	0.185	0.644	1.940
No of Parents or Children on Board	0.084	0.483	1.722

Proportion of each category make sense

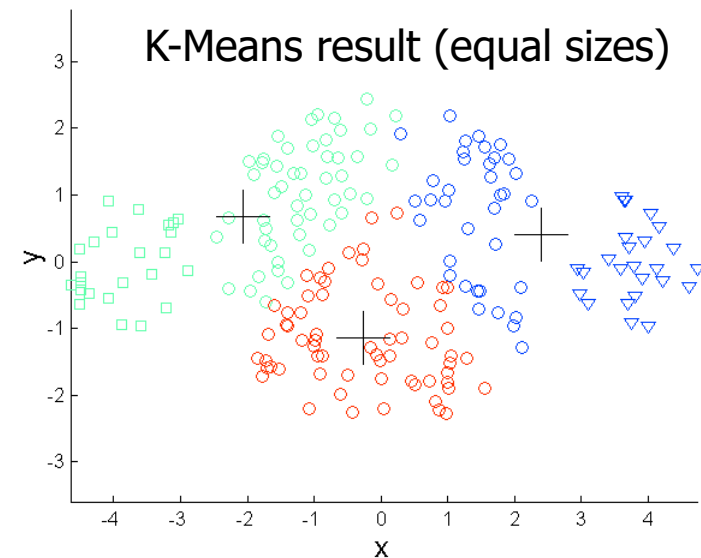
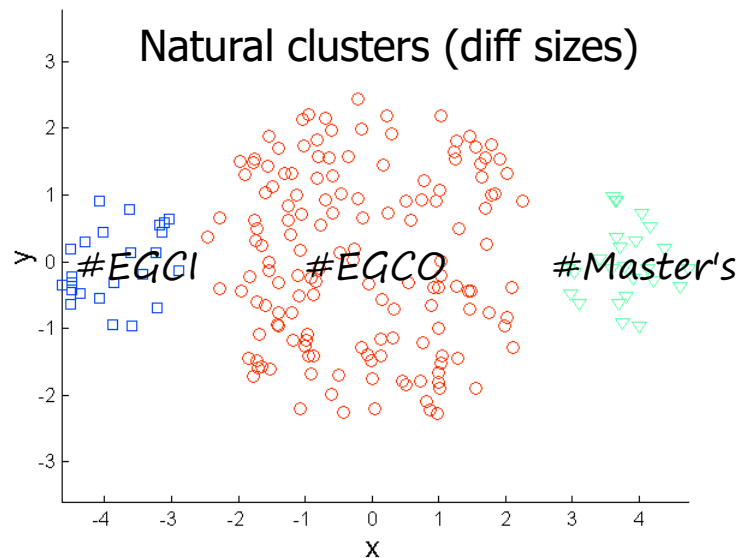
But negative values don't make sense

Row No.	id	att_1	att_2	att_3
1	count(cluster)	634.0	149.0	133.0
2	mode(Survived)	No	Yes	No
3	average(Age)	29.900901290932...	42.42826406123681	16.079141952155...
4	average(Passenger Fare)	15.216843848580...	113.764178523489...	31.172964661654...
5	mode(Sex)	Male	Female	Female
6	average(Sex = Female)	0.2602523659305...	0.55704697986577...	0.556390977443609
7	average(Sex = Male)	0.7397476340694...	0.44295302013422...	0.4436090225563...
8	mode(Passenger Class)	Third	First	Third
9	average(Passenger Class = First)	0.1388012618296...	0.97986577181208...	0.0526315789473...
10	average(Passenger Class = Second)	0.2287066246056...	0.02013422818791...	0.2706766917293...
11	average(Passenger Class = Third)	0.6324921135646...	0.0	0.6766917293233...
12	average(No of Siblings or Spouses on Board)	0.1845425867507...	0.64429530201342...	1.9398496240601...
13	average(No of Parents or Children on Board)	0.0835962145110...	0.48322147651006...	1.7218045112781...
14	cluster	cluster_0	cluster_1	cluster_2

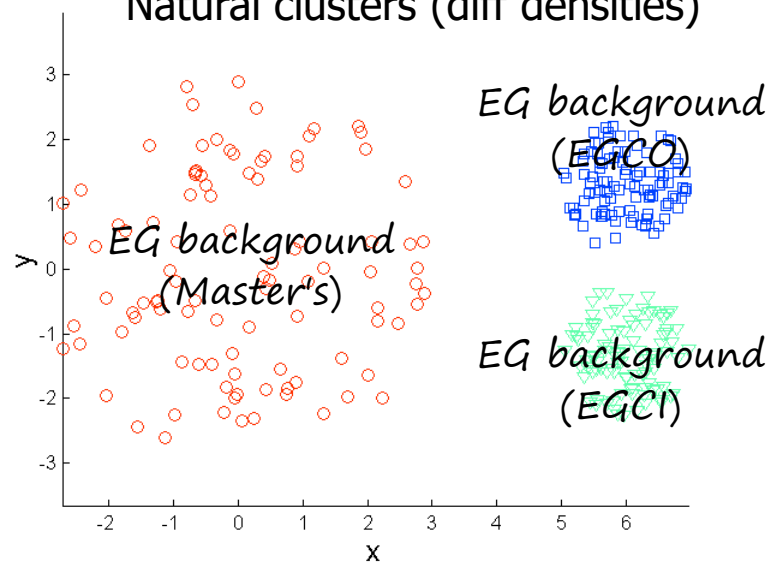
Cluster profiles after joining tables

Limitation of K-Means

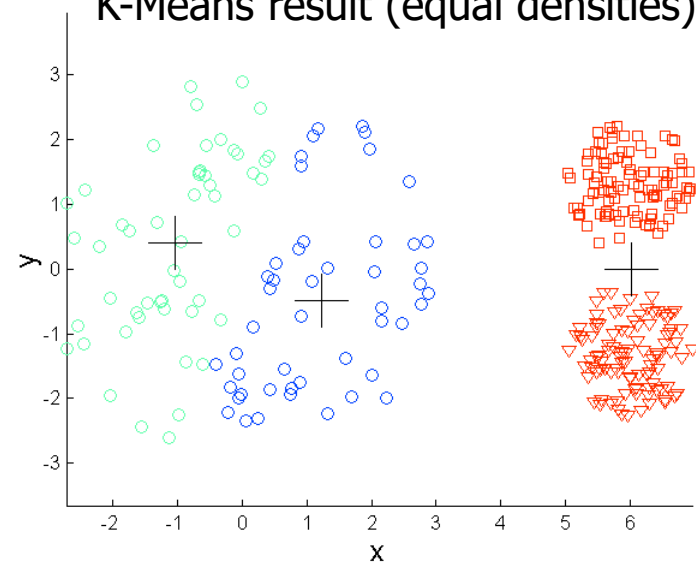
- ✦ Distortion due to outliers
 - ▶ Detect & eliminate them prior to clustering
 - ▶ Use K-Medoids instead
- ✦ Cannot find natural clusters that have different sizes, densities, or non-globular shapes



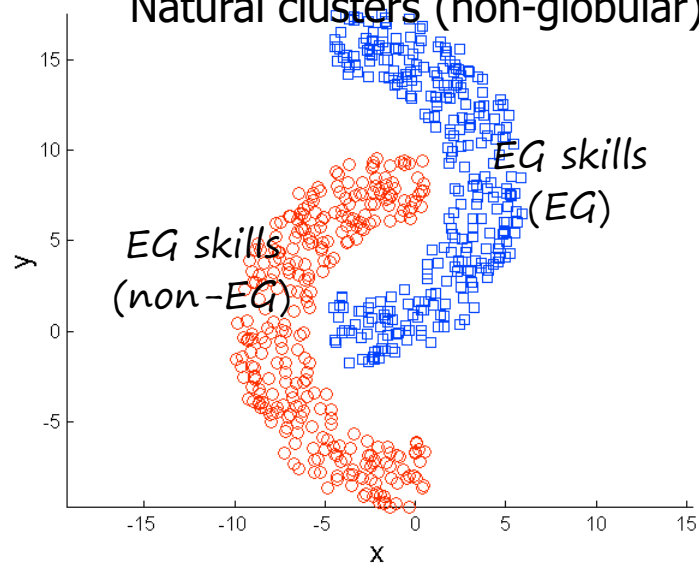
Natural clusters (diff densities)



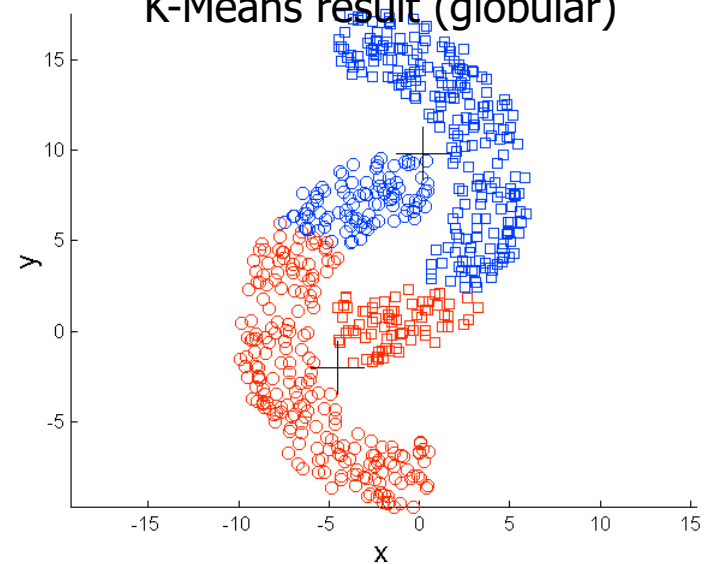
K-Means result (equal densities)



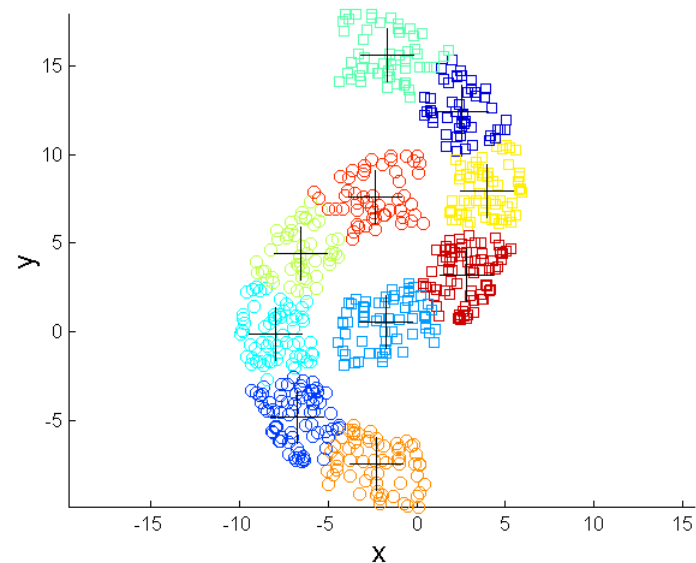
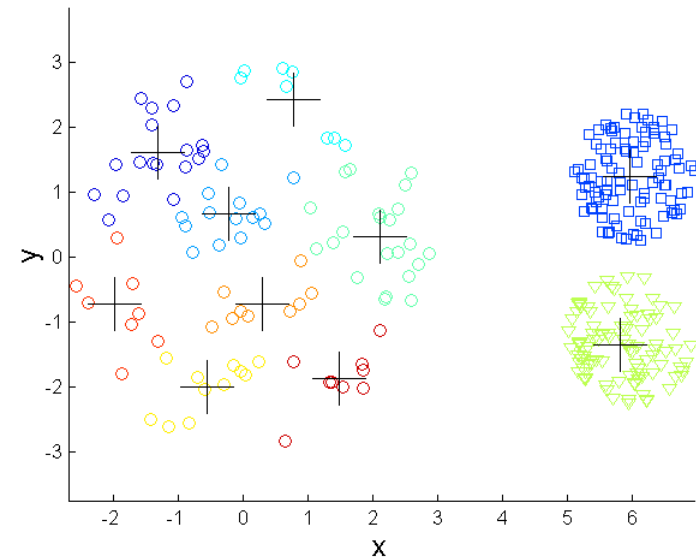
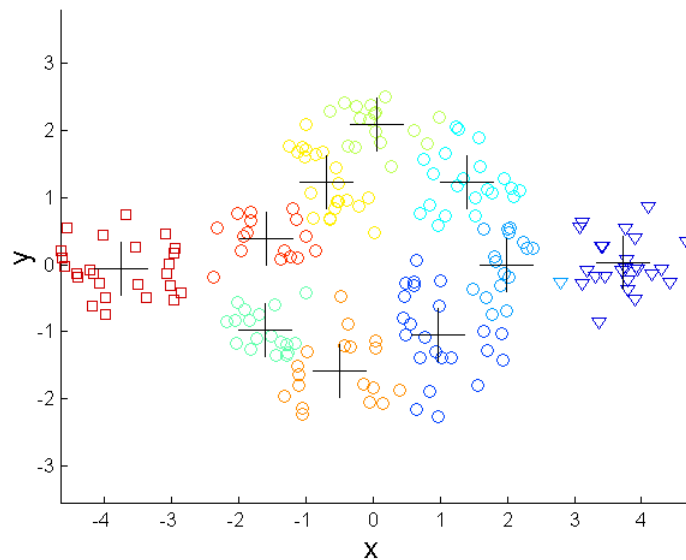
Natural clusters (non-globular)



K-Means result (globular)



Solution → increase K to obtain smaller & purer clusters. Then merge similar or related clusters



Expectation-Maximization (EM)

- ⊕ Based on **mixture model** concept
 - ▶ Each cluster is assumed to be the representative of each population
 - ▶ Each population has (normal) probability distribution or data model with parameters μ, σ^2
- ⊕ Consider 2 clusters A, B $\rightarrow$ models = $N(\mu_A, \sigma_A^2)$ and $N(\mu_B, \sigma_B^2)$
- ⊕ Given a value x , the probability that it comes from each cluster

$$P(A | x) = \frac{P(x | A) P(A)}{P(x)} ; P(B | x) = \frac{P(x | B) P(B)}{P(x)}$$

- ▶ $P(x)$ is removed so that the numerators don't sum to 1
- ▶ $P(x | A) P(A)$ is called the likelihood that x is in cluster A

⊕ Likelihood

- ▶ $P(x | A)$ is calculated from normal distribution function

$$P(x | A) = \frac{1}{\sqrt{2\pi\sigma_A^2}} e^{-\frac{(x-\mu_A)^2}{2\sigma_A^2}}$$

We used this function in Chapter 5 (Naïve Bayes)

or by using Excel function `NORM.DIST(x,  $\mu_A$ ,  $\sigma_A$ , false)`

- ▶ Total likelihood of $x = P(x | A) P(A) + P(x | B) P(B)$
If x is in the correct cluster, the likelihood should be high

⊕ Log-likelihood

- ▶ Total likelihood = the likelihoods of all points multiplied together. Log function is applied to make the calculation faster
- ▶ Clustering objective = maximize log-likelihood

⊕ Clustering steps are the same as in K-Means

Random initial μ and σ for K clustering models

For each cycle {

Expectation

For each data point X {

1. Calculate the likelihood that it comes from each cluster
2. Assign X to cluster with the highest likelihood

}

Complete 1 cycle i.e. all points are put in clusters → update μ, σ

Calculate log-likelihood for all points *Maximization*

} until stopping criteria e.g. parameters or log-likelihood is stable,
#cycles completed

RapidMiner Marketplace

Select components to install and update below. Updates to RapidMiner Studio will always be installed globally. Any global update requires administrator privileges, both during the update and the subsequent restart.

Search Updates Top Downloads Top Rated Purchased Bookmarks

smile Search

Smile 0.6.0
Smile Extension: This extension wraps functionality from the Smile library and provides them as operators.
✓ Package is up to date

Install Smile extension

Version	0.6.0
Release date	Sep 10, 2021
File size	1.3 MB
License	AGPL

This extension wraps functionality from the Smile library (<http://haifengl.github.io/smile/>) and provides them as operators.

Smile is a fast and comprehensive machine learning engine. They focus on Speed, Ease of Use, Comprehensive, Natural Language Processing and Mathematics and Statistics.

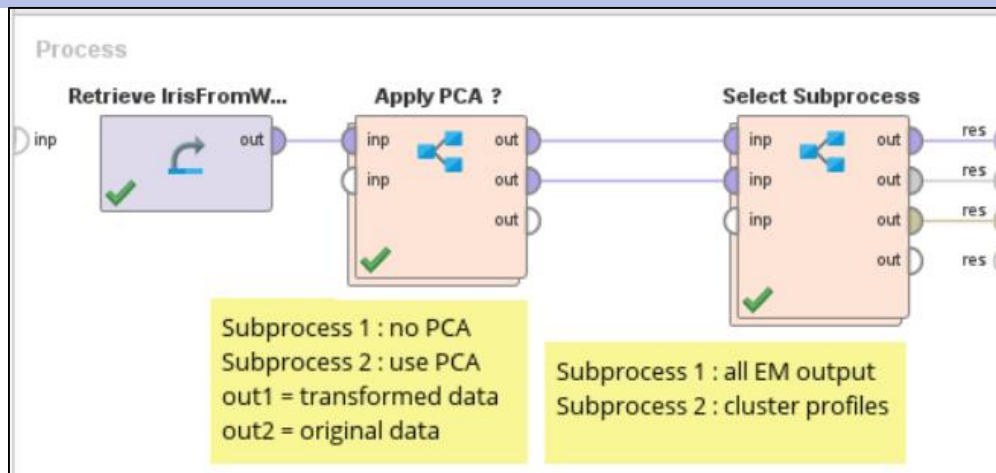
Currently the extension provides the following Operators:

- Anomaly:
 - Gaussian Mixture
- Blending:
 - t-SNE
- Cleansing:
 - Probabilistic Principal Component Analysis (PPCA)
- Clustering:
 - G-Means
- Models:
 - Parametric Probability Estimator
- Learner:

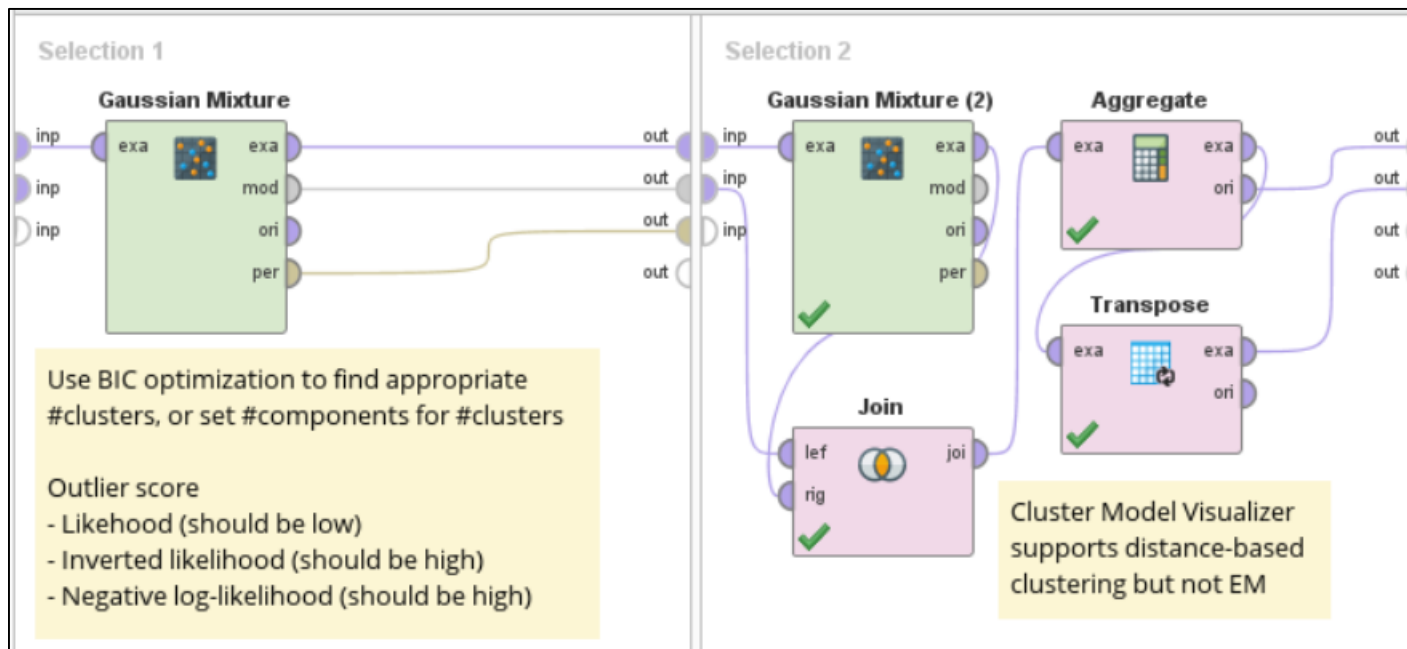
Go to [extension homepage](#)

✓ Install 0 packages ✗ Close

Example (4) : EM (Gaussian Mixture)



Smile's EM clustering outputs likelihood score of each point. Points with low likelihood are unlikely to fit any cluster & thus outliers



id	class	confidence(cluster_0)	confidence(cluster_1)	cluster	score	pc_1	pc_2
20	Iris-setosa	0.000	1.000	cluster_1	0.113	-2.587	-0.520
21	Iris-setosa	0.000	1.000	cluster_1	0.300	-2.311	-0.398
22	Iris-setosa	0.000	1.000	cluster_1	0.101	-2.543	-0.440
23	Iris-setosa	0.000	1.000	cluster_0	10.104	-3.216	-0.142
24	Iris-setosa	0.000	1.000	cluster_1	0.615	-2.303	-0.106
25	Iris-setosa	0.000	1.000	cluster_1	0.564	-2.356	0.031

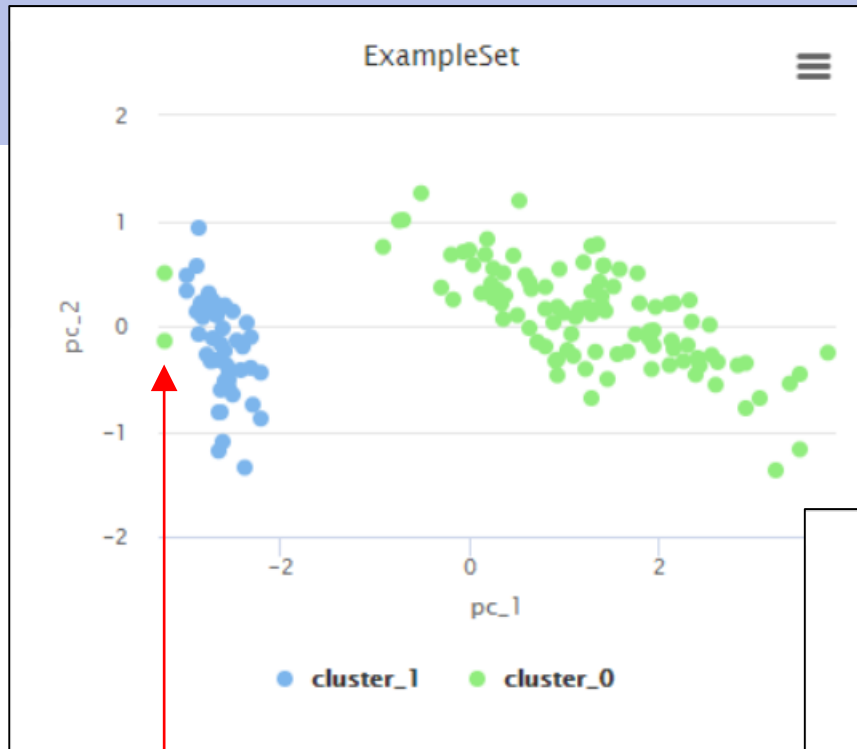
Score = inverted likelihood record #23 is likely to be outlier

With optimal #clusters = 2 BIC = 121.206 (higher BIC is better)

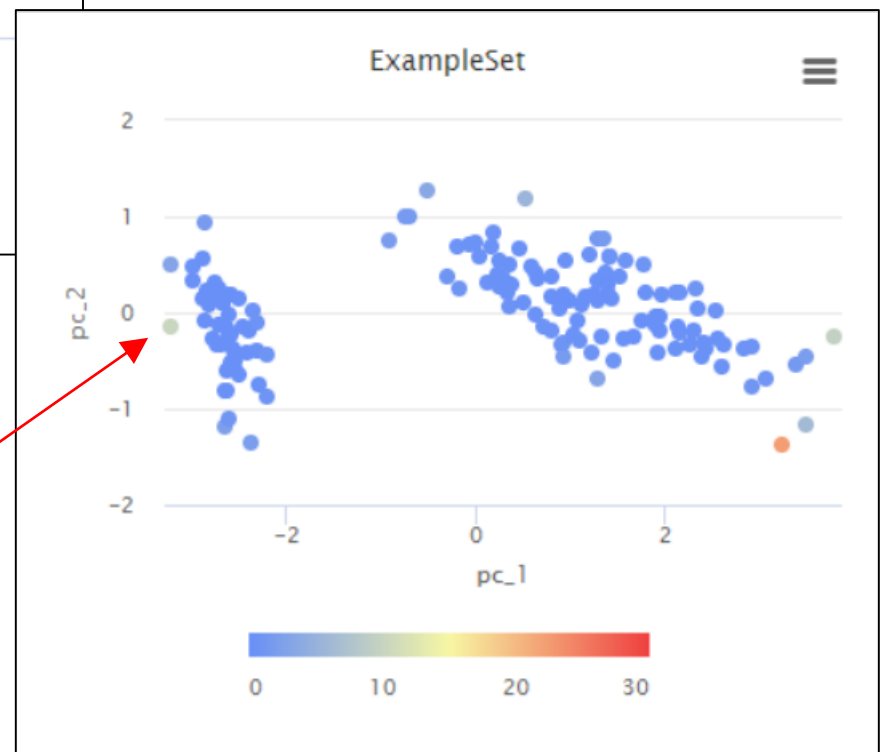
But if we set #clusters = 3 BIC = 81.943

Use visualization to check how data points are clustered

Build cluster profiles from original attributes

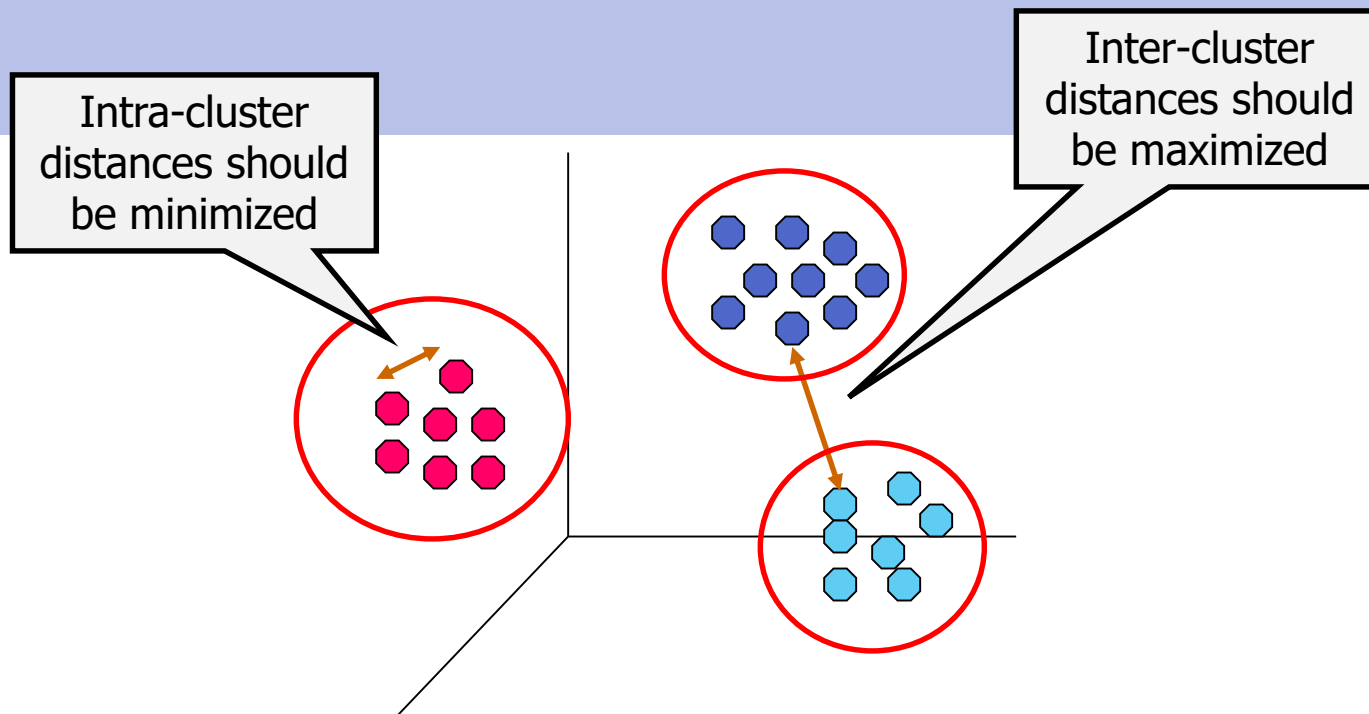


This point is assigned to cluster 0 (but it shouldn't be). It's inverted likelihood is high so it may be an outlier



Cluster Evaluation

- ⊕ Unlike classification, evaluating results from clustering is tricky
 - ▶ Its main purpose is understanding data, not prediction
 - ▶ No ground truth to compare whether cluster assignment is correct or not correct
- ⊕ **Unsupervised evaluation**
 - ▶ Don't use external information
 - ▶ Evaluate how well the results fit the data
 - ▶ Performance metrics can vary from method to method
 - Distance-based → SSE, DBI, cohesion & separation metrics
 - Probability-based → log-likelihood, BIC



⊕ **Cohesion** : measure how closely related objects in the same clusters are

► WSS = within cluster SSE = total intra-cluster distance

$$WSS = \sum_{\forall \text{ clusters}} \left(\sum_{\forall \text{ points in cluster}} (d \text{ from point to centroid})^2 \right)$$

Same as SSE function in Slide 11

⊕ **Separation** : measure how well separated clusters are from each other

- ▶ BSS = between cluster SSE = total inter-cluster distance

$$BSS = \sum_{\forall \text{ cluster } i} \| \text{cluster}_i \| (d \text{ from centroid}_i \text{ to centroid}_{all})^2$$

$\| \text{cluster}_i \|$ = size of cluster i ; centroid_{all} = centroid of whole data set

⊕ **TSS = total SSE = WSS + BSS**

- ▶ TSS is constant for a given data set
- ▶ When comparing clustering results, we prefer high cohesion (i.e. low WSS) & high separation (i.e. high BSS)

RapidMiner doesn't output these values but we can use DBI (already the trade-off between cohesion and separation) instead

Example : clustering result (1)

$$\text{Centroid}_1 = (1.33, 2)$$

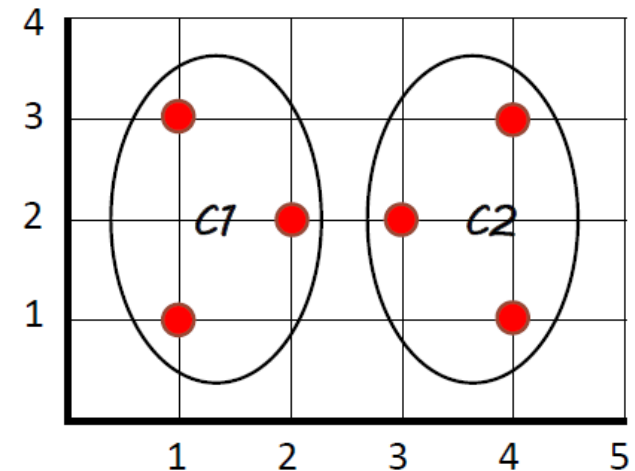
$$\text{Centroid}_2 = (3.67, 2)$$

$$\text{Centroid}_{\text{all}} = (2.5, 2) \quad \text{averaged from 6 points}$$

$$\begin{aligned} \text{WSS} &= ((1 - 1.33)^2 + (1 - 2)^2) + \\ &\quad ((1 - 1.33)^2 + (3 - 2)^2) + \\ &\quad ((2 - 1.33)^2 + (2 - 2)^2) + \\ &\quad ((3 - 3.67)^2 + (2 - 2)^2) + ((4 - 3.67)^2 + (1 - 2)^2) + ((4 - 3.67)^2 + (3 - 2)^2) \\ &= 2.6667 + 2.6667 = \underline{5.3334} \end{aligned}$$

$$\begin{aligned} \text{BSS} &= \left(3 \times ((1.33 - 2.5)^2 + (2 - 2)^2) \right) + \left(3 \times ((3.67 - 2.5)^2 + (2 - 2)^2) \right) \\ &= 4.1067 + 4.1067 = \underline{8.2134} \end{aligned}$$

$$\text{TSS} = 5.3334 + 8.2134 = 13.5468$$



Example : clustering result (2)

$$\text{Centroid}_1 = (1, 2)$$

$$\text{Centroid}_2 = (2.5, 2)$$

$$\text{Centroid}_3 = (4, 2)$$

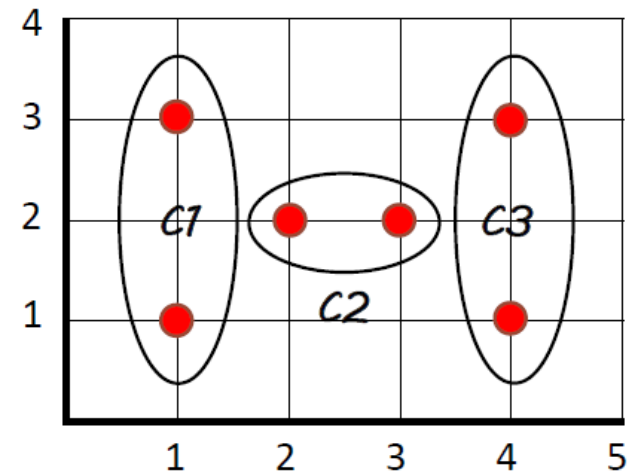
$$\text{Centroid}_{\text{all}} = (2.5, 2) \quad \text{averaged from 6 points}$$

$$\begin{aligned} \text{WSS} = & ((1-1)^2 + (1-2)^2) + ((1-1)^2 + (3-2)^2) + \\ & ((2-2.5)^2 + (2-2)^2) + ((3-2.5)^2 + (3-2)^2) + \\ & ((4-4)^2 + (1-2)^2) + ((4-4)^2 + (3-2)^2) \\ = & 2 + 0.5 + 2 = \underline{4.5} \end{aligned}$$

$$\begin{aligned} \text{BSS} = & \left(2 \times ((1-2.5)^2 + (2-2)^2) \right) + \left(2 \times ((2.5-2.5)^2 + (2-2)^2) \right) + \\ & \left(2 \times ((4-2.5)^2 + (2-2)^2) \right) \\ = & 4.5 + 0 + 4.5 = \underline{9} \end{aligned}$$

$$\text{TSS} = 4.5 + 9 = 13.5 \quad \text{slightly differ from result (1) due to floating point rounding}$$

Result (2) yields lower WSS (i.e. better cohesion) and higher BSS (i.e. better separation)



Cluster Evaluation (cont.)

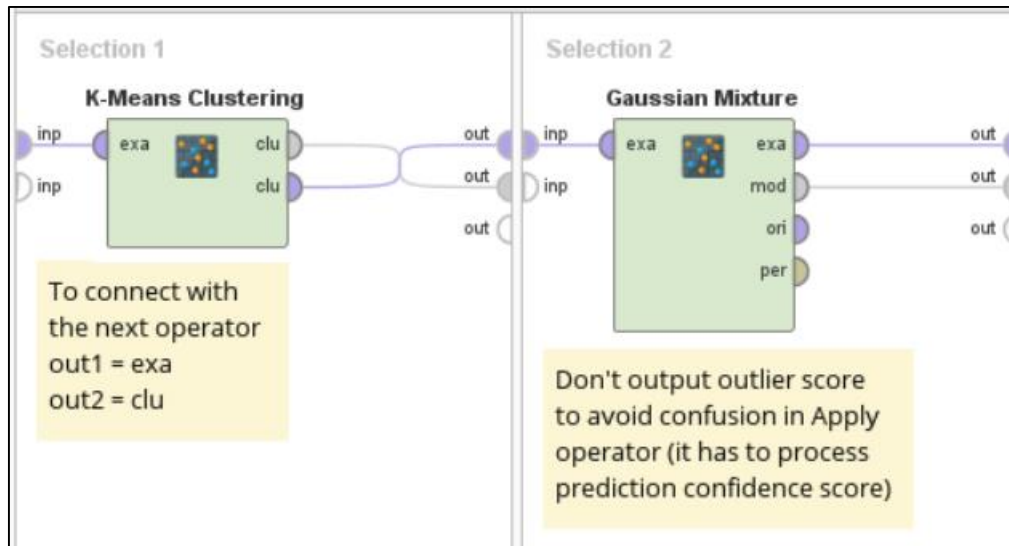
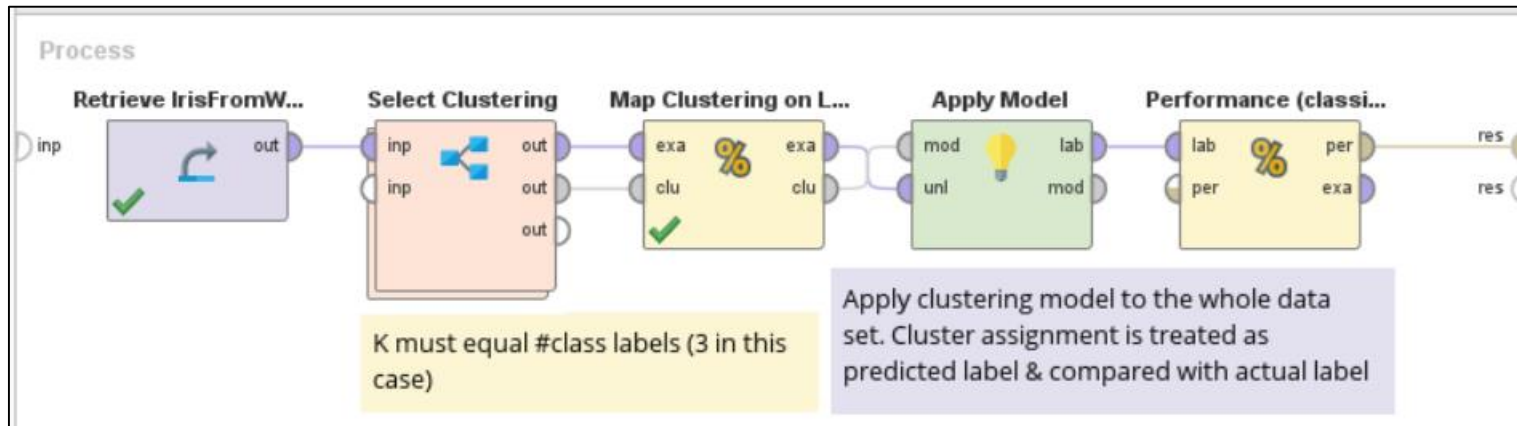
⊕ Supervised evaluation

- ▶ Use external info as guideline of what natural clusters should be
- ▶ Study domain background. Does cluster assignment make any sense ?
- ▶ If class labels are available → treat cluster assignment as prediction against class labels
 - But class labels are only guideline, not the ground truth. We can only say whether cluster assignment agrees with them
- ▶ Or use classification models (e.g. rules) to explain cluster assignment

⊕ Relative evaluation

- ▶ Compare results from different clustering methods

Example (5) : map cluster assignment & class labels

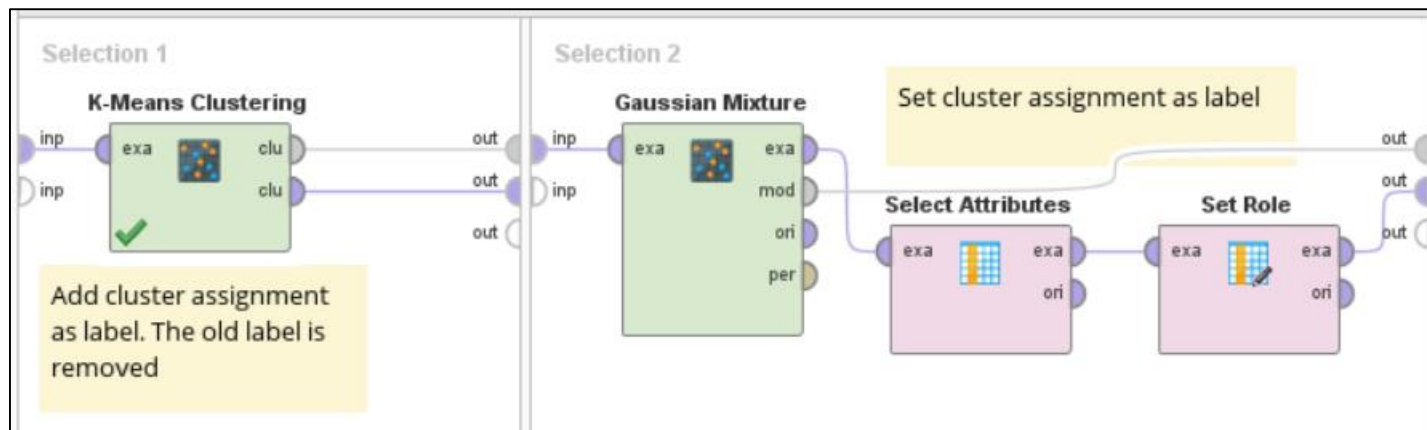
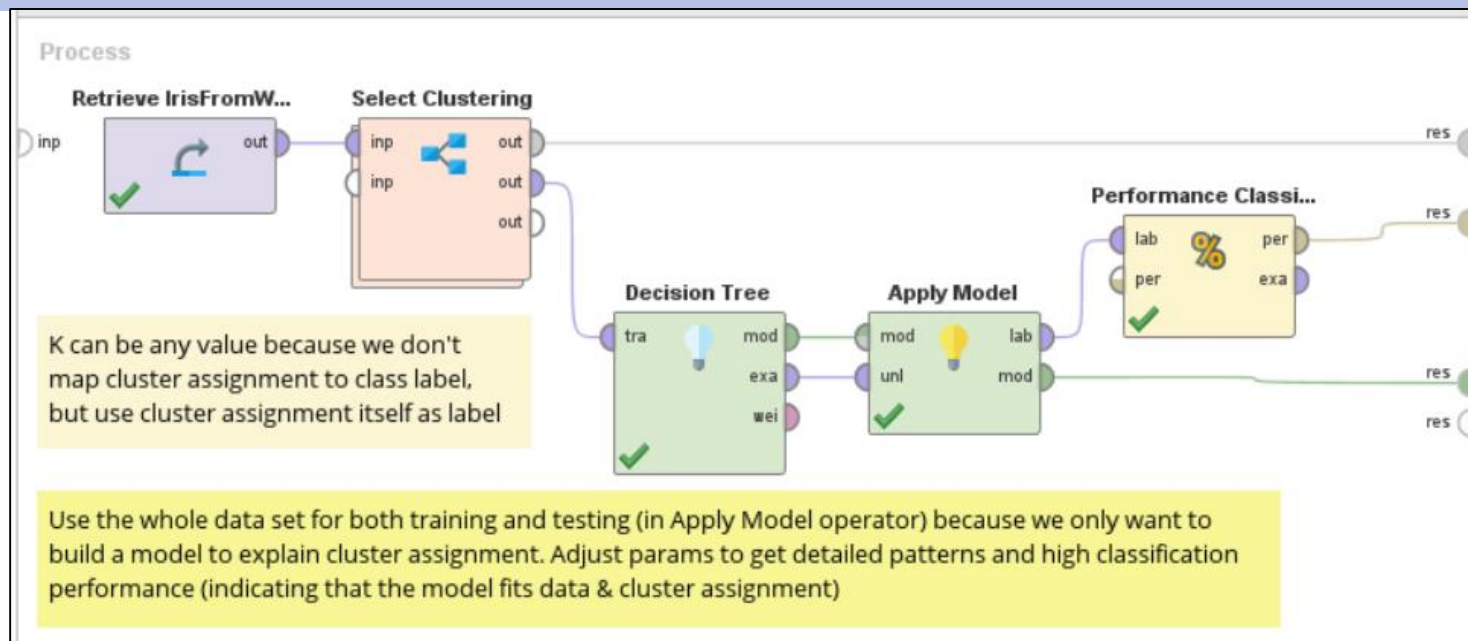


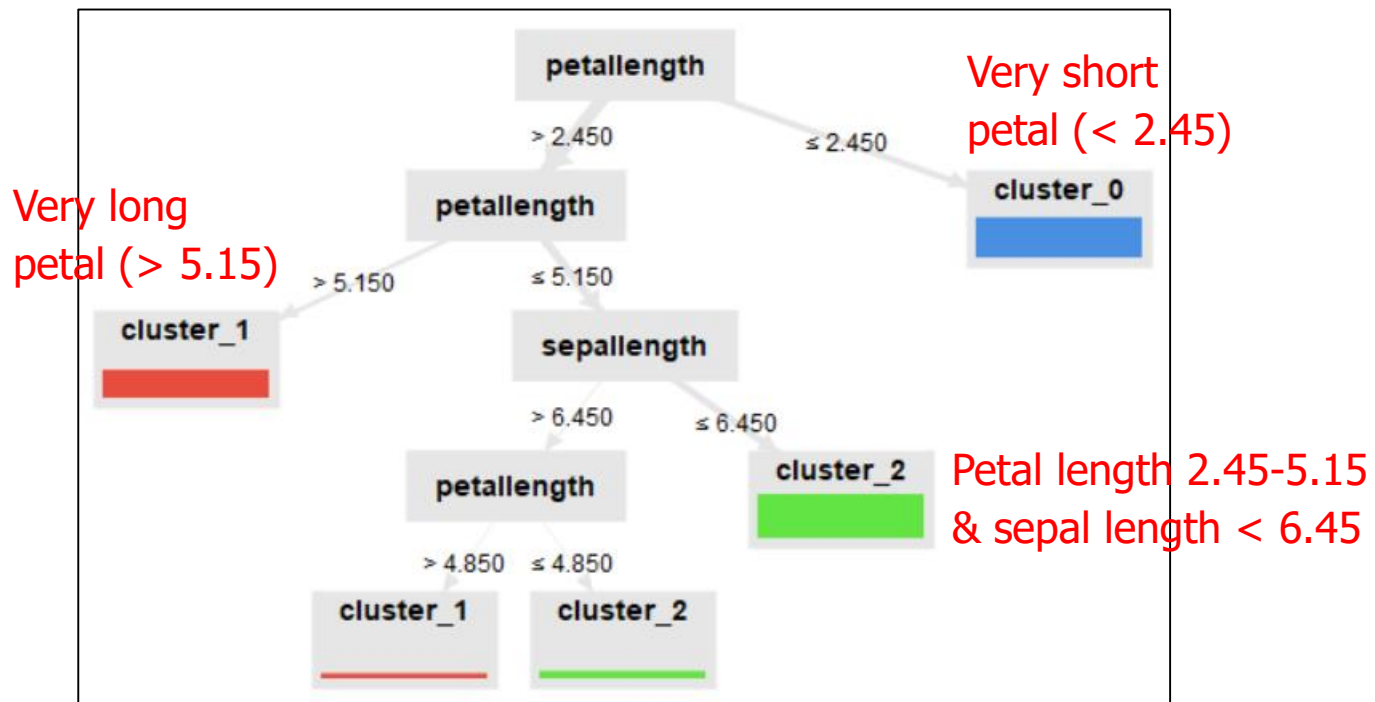
K-Means accuracy = 88.67%

EM accuracy = 92.67%

Cluster assignment by EM is more in agreement with class labels (if we believe that class labels are good indicators of natural clusters)

Example (6) : classification model on cluster assignment





Accuracy = 100% → tree perfectly fits the data
 Read conditions that lead to each cluster assignment

Summary

- ⊕ Clustering steps in K-Means, K-Medoids, and EM are similar. They all require K (#clusters) from users
- ⊕ K-Means and K-Medoids
 - ▶ Calculate distance between each data point and centroids
 - ▶ Cluster members are points around centroid → clusters are usually in globular shape with equal size and equal density
 - ▶ Objective = minimize SSE or DBI
 - ▶ K-Means is more sensitive to outliers than K-Medoids
- ⊕ EM
 - ▶ Calculate the likelihood that each data point belongs to each cluster
 - ▶ Cluster members don't need to be around any particular point → they can have any shape, any size, any density
 - ▶ Objective = maximize log-likelihood or BIC